# C++ Standard Library Issues to be moved in Wrocław, Nov. 2024

Doc. no. P3504R0

Date: 2024-11-18

Audience: WG21
Reply to: Jonathan Wakely <lwgchair@gmail.com>

- Ready Issues
- Tentatively Ready Issues

## Ready Issues

---

### 3436. `std::construct_at` should support arrays

**Section:** 26.11.8 [specialized.construct] **Status:** Ready **Submitter:** Jonathan Wakely **Opened:** 2020-04-29 **Last modified:** 2024-06-24

**Priority:** 2

**View other** active issues in [specialized.construct].

**View all other** issues in [specialized.construct].

**Discussion:**

`std::construct_at` is ill-formed for array types, because the type of the `new`-expression is `T` not `T*` so it cannot be converted to the return type.

In C++17 `allocator_traits::construct` did work for arrays, because it returns `void` so there is no ill-formed conversion. On the other hand, in C++17 `allocator_traits::destroy` didn't work for arrays, because `p->~T()` isn't valid.

In C++20 `allocator_traits::destroy` does work, because `std::destroy_at` treats arrays specially, but `allocator_traits::construct` no longer works because it uses `std::construct_at`.

It seems unnecessary and/or confusing to remove support for arrays in `construct` when we're adding it in `destroy`.

I suggest that `std::construct_at` should also handle arrays. It might be reasonable to restrict that support to the case where `sizeof...(Args) == 0`, if supporting parenthesized aggregate-initialization is not desirable in `std::construct_at`.

*[2020-05-09; Reflector prioritization]*

Set priority to 2 after reflector discussions.

*[2021-01-16; Zhihao Yuan provides wording]*

**Previous resolution [SUPERSEDED]:**

This wording is relative to N4878.

1. Modify 26.11.8 [specialized.construct] as indicated:

```
template<class T, class... Args>
  constexpr T* construct_at(T* location, Args&&... args);

namespace ranges {
  template<class T, class... Args>
    constexpr T* construct_at(T* location, Args&&... args);
}
```

-1- *Constraints:* The expression `::new (declval<void*>()) T(declval<Args>()...)` is well-formed when treated as an unevaluated operand.

-2- *Effects:* Equivalent to:

```
return auto ptr = ::new (voidify(*location)) T(std::forward<Args>(args)...);
if constexpr (is_array_v<T>)
  return launder(location);
else
  return ptr;
```

*[2021-12-07; Zhihao Yuan comments and provides improved wording]*

The previous PR allows constructing arbitrary number of elements when `T` is an array of unknown bound:

```
extern int a[];
std::construct_at(&a, 0, 1, 2);
```

and leads to a UB.

**Previous resolution [SUPERSEDED]:**

This wording is relative to N4901.

1. Modify 26.11.8 [specialized.construct] as indicated:

```
template<class T, class... Args>
  constexpr T* construct_at(T* location, Args&&... args);

namespace ranges {
  template<class T, class... Args>
    constexpr T* construct_at(T* location, Args&&... args);
}
```

-1- *Constraints:* The expression `::new (declval<void*>()) T(declval<Args>()...)` is well-formed when treated as an unevaluated operand (7.2.3 [expr.context]) and `is_unbounded_array_v<T>` is `false`.

-2- *Effects:* Equivalent to:

```
return auto ptr = ::new (voidify(*location)) T(std::forward<Args>(args)...);
if constexpr (is_array_v<T>)
  return launder(location);
else
  return ptr;
```

*[2024-03-18; Jonathan provides new wording]*

During Core review in Varna, Hubert suggested creating `T[1]` for the array case.

**Previous resolution [SUPERSEDED]:**

This wording is relative to N4971.

1. Modify 26.11.8 [specialized.construct] as indicated:

```
template<class T, class... Args>
  constexpr T* construct_at(T* location, Args&&... args);

namespace ranges {
  template<class T, class... Args>
    constexpr T* construct_at(T* location, Args&&... args);
}
```

-1- *Constraints:* `is_unbounded_array_v<T>` is `false`. The expression `::new (declval<void*>()) T(declval<Args>()...)` is well-formed when treated as an unevaluated operand (7.2.3 [expr.context]).

-2- *Effects:* Equivalent to:

```
if constexpr (is_array_v<T>)
  return ::new (voidify(*location)) T[1]{{std::forward<Args>(args)...}};
else
  return ::new (voidify(*location)) T(std::forward<Args>(args)...);
```

*[St. Louis 2024-06-24; Jonathan provides improved wording]*

Why not support unbounded arrays, deducing the bound from `sizeof...(Args)`?
JW: There's no motivation to support that here in `construct_at`. It isn't possible to create unbounded arrays via allocators, nor via any of the `uninitialized_xxx` algorithms. Extending `construct_at` that way seems like a design change, not restoring support for something that used to work with allocators and then got broken in C++20.

Tim observed that the proposed resolution is ill-formed if `T` has an explicit default constructor. Value-initialization would work for that case, and there seems to be little motivation for supplying arguments to initialize the array. In C++17 the `allocator_traits::construct` case only supported value-initialization.

*[St. Louis 2024-06-24; move to Ready.]*

**Proposed resolution:**

This wording is relative to N4981.

1. Modify 26.11.8 [specialized.construct] as indicated:

```
template<class T, class... Args>
  constexpr T* construct_at(T* location, Args&&... args);

namespace ranges {
  template<class T, class... Args>
    constexpr T* construct_at(T* location, Args&&... args);
}
```

-1- *Constraints:* `is_unbounded_array_v<T>` is `false`. The expression `::new (declval<void*>()) T(declval<Args>()...)` is well-formed when treated as an unevaluated operand (7.2.3 [expr.context]).

-?- *Mandates*: If `is_array_v<T>` is `true`, `sizeof...(Args)` is zero.

-2- *Effects:* Equivalent to:

```
if constexpr (is_array_v<T>)
  return ::new (voidify(*location)) T[1]();
else
  return ::new (voidify(*location)) T(std::forward<Args>(args)...);
```

---

## 3899. `co_yield`ing elements of an lvalue `generator` is unnecessarily inefficient

**Section:** 25.8.5 [coro.generator.promise] **Status:** Ready **Submitter:** Tim Song **Opened:** 2023-03-04 **Last modified:** 2024-06-28

**Priority:** 3

**View other** active issues in [coro.generator.promise].

**View all other** issues in [coro.generator.promise].

**Discussion:**

Consider:

```
std::generator<int> f();
std::generator<int> g() {
    auto gen = f();
    auto gen2 = f();
    co_yield std::ranges::elements_of(std::move(gen));   // #1
    co_yield std::ranges::elements_of(gen2);             // #2
    // other stuff
}
```

Both #1 and #2 compile. The differences are:

- #2 is significantly less efficient (it uses the general overload of `yield_value`, so it creates a new coroutine frame and doesn't do symmetric transfer into `gen2`'s coroutine)
- the coroutine frame of `gen` and `gen2` are destroyed at different times: `gen`'s frame is destroyed at the end of #1, but `gen2`'s is not destroyed until the closing brace.

But as far as the user is concerned, neither `gen` nor `gen2` is usable after the `co_yield`. In both cases the only things you can do with the objects are:

- destroying them;
- assigning to them;
- call `end()` on them to get a copy of `default_sentinel`.

We could make #2 ill-formed, but that seems unnecessary: there is no meaningful difference between `generator` and any other single-pass input range (or a `generator` with a different yielded type that has to go through the general overload) in this regard. We should just make #2 do the efficient thing too.

*[2023-03-22; Reflector poll]*

Set priority to 3 after reflector poll.

*[St. Louis 2024-06-28; move to Ready]*

**Proposed resolution:**

This wording is relative to N4928.

1. Modify 25.8.5 [coro.generator.promise] as indicated:

```
namespace std {
  template<class Ref, class V, class Allocator>
  class generator<Ref, V, Allocator>::promise_type {
  public:
    […]
    auto yield_value(const remove_reference_t<yielded>& lval)
      requires is_rvalue_reference_v<yielded> &&
        constructible_from<remove_cvref_t<yielded>, const remove_reference_t<yielded>&>;

    template<class R2, class V2, class Alloc2, class Unused>
      requires same_as<typename generator<R2, V2, Alloc2>::yielded, yielded>
        auto yield_value(ranges::elements_of<generator<R2, V2, Alloc2>&&, Unused> g) noexcept;
    template<class R2, class V2, class Alloc2, class Unused>
      requires same_as<typename generator<R2, V2, Alloc2>::yielded, yielded>
        auto yield_value(ranges::elements_of<generator<R2, V2, Alloc2>&, Unused> g) noexcept;

    template<ranges::input_range R, class Alloc>
      requires convertible_to<ranges::range_reference_t<R>, yielded>
        auto yield_value(ranges::elements_of<R, Alloc> r) noexcept;
```

```
    […]
  };
}
```

[…]

```
template<class R2, class V2, class Alloc2, class Unused>
  requires same_as<typename generator<R2, V2, Alloc2>::yielded, yielded>
  auto yield_value(ranges::elements_of<generator<R2, V2, Alloc2>&&, Unused> g) noexcept;
template<class R2, class V2, class Alloc2, class Unused>
  requires same_as<typename generator<R2, V2, Alloc2>::yielded, yielded>
  auto yield_value(ranges::elements_of<generator<R2, V2, Alloc2>&, Unused> g) noexcept;
```

-10- *Preconditions*: A handle referring to the coroutine whose promise object is `*this` is at the top of `*active_` of some generator object `x`. The coroutine referred to by `g.range.coroutine_` is suspended at its initial suspend point.

-11- *Returns:* An awaitable object of an unspecified type (7.6.2.4 [expr.await]) into which `g.range` is moved, whose member `await_ready` returns `false`, whose member `await_suspend` pushes `g.range.coroutine_` into `*x.active_` and resumes execution of the coroutine referred to by `g.range.coroutine_`, and whose member `await_resume` evaluates `rethrow_exception(except_)` if `bool(except_)` is `true`. If `bool(except_)` is `false`, the `await_resume` member has no effects.

-12- *Remarks*: A *yield-expression* that calls ~~this function~~one of these functions has type `void` (7.6.17 [expr.yield]).

---

## 3900. The `allocator_arg_t` overloads of `generator::promise_type::operator new` should not be constrained

**Section:** 25.8.5 [coro.generator.promise] **Status:** Ready **Submitter:** Tim Song **Opened:** 2023-03-04 **Last modified:** 2024-06-28

**Priority:** 3

**View other** active issues in [coro.generator.promise].

**View all other** issues in [coro.generator.promise].

**Discussion:**

When the allocator is not type-erased, the `allocator_arg_t` overloads of `generator::promise_type::operator new` are constrained on `convertible_to<const Alloc&, Allocator>`. As a result, if the the allocator is default-constructible (like `polymorphic_allocator` is) but the user accidentally provided a wrong type (say, `memory_resource&` instead of `memory_resource*`), their code will silently fall back to using a default-constructed allocator. It would seem better to take the tag as definitive evidence of the user's intent to supply an allocator for the coroutine, and error out if the supplied allocator cannot be used.

This change does mean that the user cannot deliberately pass an incompatible allocator (preceded by an `std::allocator_arg_t` tag) for their own use inside the coroutine, but that sort of API seems fragile and confusing at best, since the usual case is that allocators so passed *will* be used by `generator`.

*[2023-03-22; Reflector poll]*

Set priority to 3 after reflector poll.

*[St. Louis 2024-06-28; move to Ready]*

**Proposed resolution:**

This wording is relative to N4928.

1. Modify 25.8.5 [coro.generator.promise] as indicated:

```
namespace std {
  template<class Ref, class V, class Allocator>
  class generator<Ref, V, Allocator>::promise_type {
  public:
    […]
    void* operator new(size_t size)
      requires same_as<Allocator, void> || default_initializable<Allocator>;

    template<class Alloc, class... Args>
      requires same_as<Allocator, void> || convertible_to<const Alloc&, Allocator>
        void* operator new(size_t size, allocator_arg_t, const Alloc& alloc, const Args&...);

    template<class This, class Alloc, class... Args>
      requires same_as<Allocator, void> || convertible_to<const Alloc&, Allocator>
        void* operator new(size_t size, const This&, allocator_arg_t, const Alloc& alloc,
                           const Args&...);
    […]
  };
}
```

[…]

```
void* operator new(size_t size)
  requires same_as<Allocator, void> || default_initializable<Allocator>;

template<class Alloc, class... Args>
  requires same_as<Allocator, void> || convertible_to<const Alloc&, Allocator>
  void* operator new(size_t size, allocator_arg_t, const Alloc& alloc, const Args&...);

template<class This, class Alloc, class... Args>
  requires same_as<Allocator, void> || convertible_to<const Alloc&, Allocator>
```

```
void* operator new(size_t size, const This&, allocator_arg_t, const Alloc& alloc,
                   const Args&...);
```

-17- Let `A` be

(17.1) — `Allocator`, if it is not `void`,

(17.2) — `Alloc` for the overloads with a template parameter `Alloc`, or

(17.3) — `allocator<void>` otherwise.

Let `B` be `allocator_traits<A>::template rebind_alloc<U>` where `U` is an unspecified type whose size and alignment are both `__STDCPP_DEFAULT_NEW_ALIGNMENT__`.

-18- *Mandates*: `allocator_traits<B>::pointer` is a pointer type. For the overloads with a template parameter `Alloc`, `same_as<Allocator, void> || convertible_to<const Alloc&, Allocator>` is modeled.

-19- *Effects*: Initializes an allocator `b` of type `B` with `A(alloc)`, for the overloads with a function parameter `alloc`, and with `A()` otherwise. Uses `b` to allocate storage for the smallest array of `U` sufficient to provide storage for a coroutine state of size `size`, and unspecified additional state necessary to ensure that `operator delete` can later deallocate this memory block with an allocator equal to `b`.

-20- *Returns:* A pointer to the allocated storage.

---

## 3918. `std::uninitialized_move/_n` and guaranteed copy elision

**Section:** 26.11.6 [uninitialized.move] **Status:** Ready **Submitter:** Jiang An **Opened:** 2023-04-04 **Last modified:** 2024-06-26

**Priority:** 3

**Discussion:**

Currently `std::move` is unconditionally used in `std::uninitialized_move` and `std::uninitialized_move_n`, which may involve unnecessary move construction if dereferencing the input iterator yields a prvalue.

The status quo was mentioned in paper issue #975, but no further process is done since then.

*[2023-06-01; Reflector poll]*

Set priority to 3 after reflector poll. Send to LEWG.

"P2283 wants to remove guaranteed elision here." "Poorly motivated, not clear anybody is using these algos with proxy iterators." "Consider using `iter_move` in the move algos."

**Previous resolution [SUPERSEDED]:**

This wording is relative to N4944.

1. Modify 26.11.1 [specialized.algorithms.general] as indicated:

-3- Some algorithms specified in 26.11 [specialized.algorithms] make use of the following exposition-only functions ~~*voidify*~~:

```
template<class T>
  constexpr void* voidify(T& obj) noexcept {
    return addressof(obj);
  }

template<class I>
  decltype(auto) deref-move(const I& it) {
    if constexpr (is_lvalue_reference_v<decltype(*it)>)
      return std::move(*it);
    else
      return *it;
  }
```

2. Modify 26.11.6 [uninitialized.move] as indicated:

```
template<class InputIterator, class NoThrowForwardIterator>
  NoThrowForwardIterator uninitialized_move(InputIterator first, InputIterator last,
                                            NoThrowForwardIterator result);
```

-1- *Preconditions*: `result + [0, (last - first))` does not overlap with `[first, last)`.

-2- *Effects*: Equivalent to:

```
for (; first != last; (void)++result, ++first)
  ::new (voidify(*result))
    typename iterator_traits<NoThrowForwardIterator>::value_type(~~std::move(~~*deref-move(first));
return result;
```

[…]

```
template<class InputIterator, class Size, class NoThrowForwardIterator>
  pair<InputIterator, NoThrowForwardIterator>
    uninitialized_move_n(InputIterator first, Size n, NoThrowForwardIterator result);
```

-6- *Preconditions*: `result + [0, n)` does not overlap with `first + [0, n)`.

-7- *Effects*: Equivalent to:

```
for (; n > 0; ++result,(void) ++first, --n)
  ::new (voidify(*result))
    typename iterator_traits<NoThrowForwardIterator>::value_type(std::move(*deref-move(first));
return {first, result};
```

*[2024-03-22; Tokyo: Jonathan updates wording after LEWG review]*

LEWG agrees it would be good to do this. Using `iter_move` was discussed, but it was noted that the versions of these algos in the `ranges` namespace already use it and introducing `ranges::iter_move` into the non-ranges versions wasn't desirable. It was observed that the proposed *deref-move* has a `const I&` parameter which would be ill-formed for any iterator with a non-const `operator*` member. Suggested removing the const and recommended LWG to accept the proposed resolution.

**Previous resolution [SUPERSEDED]:**

This wording is relative to N4971.

1. Modify 26.11.1 [specialized.algorithms.general] as indicated:

-3- Some algorithms specified in 26.11 [specialized.algorithms] make use of the following exposition-only functions ~~*voidify*~~:

```
template<class T>
  constexpr void* voidify(T& obj) noexcept {
    return addressof(obj);
  }

template<class I>
  decltype(auto) deref-move(I& it) {
    if constexpr (is_lvalue_reference_v<decltype(*it)>)
      return std::move(*it);
    else
      return *it;
  }
```

2. Modify 26.11.6 [uninitialized.move] as indicated:

```
template<class InputIterator, class NoThrowForwardIterator>
  NoThrowForwardIterator uninitialized_move(InputIterator first, InputIterator last,
                                            NoThrowForwardIterator result);
```

-1- *Preconditions*: `result + [0, (last - first))` does not overlap with `[first, last)`.

-2- *Effects*: Equivalent to:

```
for (; first != last; (void)++result, ++first)
  ::new (voidify(*result))
    typename iterator_traits<NoThrowForwardIterator>::value_type(std::move(*deref-move(first));
return result;
```

[…]

```
template<class InputIterator, class Size, class NoThrowForwardIterator>
  pair<InputIterator, NoThrowForwardIterator>
    uninitialized_move_n(InputIterator first, Size n, NoThrowForwardIterator result);
```

-6- *Preconditions*: `result + [0, n)` does not overlap with `first + [0, n)`.

-7- *Effects*: Equivalent to:

```
for (; n > 0; ++result,(void) ++first, --n)
  ::new (voidify(*result))
    typename iterator_traits<NoThrowForwardIterator>::value_type(std::move(*deref-move(first));
return {first, result};
```

*[St. Louis 2024-06-24; revert P/R and move to Ready]*

Tim observed that the iterator requirements require all iterators to be const-dereferenceable, so there was no reason to remove the const. Restore the original resolution and move to Ready.

**Proposed resolution:**

This wording is relative to N4971.

1. Modify 26.11.1 [specialized.algorithms.general] as indicated:

-3- Some algorithms specified in 26.11 [specialized.algorithms] make use of the following exposition-only functions ~~*voidify*~~:

```
template<class T>
  constexpr void* voidify(T& obj) noexcept {
    return addressof(obj);
  }

template<class I>
  decltype(auto) deref-move(I& it) {
    if constexpr (is_lvalue_reference_v<decltype(*it)>)
      return std::move(*it);
    else
      return *it;
  }
```

2. Modify 26.11.6 [uninitialized.move] as indicated:

```
template<class InputIterator, class NoThrowForwardIterator>
  NoThrowForwardIterator uninitialized_move(InputIterator first, InputIterator last,
                                            NoThrowForwardIterator result);
```

-1- *Preconditions*: `result + [0, (last - first))` does not overlap with `[first, last)`.

-2- *Effects*: Equivalent to:

```
for (; first != last; (void)++result, ++first)
  ::new (voidify(*result))
    typename iterator_traits<NoThrowForwardIterator>::value_type(std::move(*deref-move(first));
return result;
```

[…]

```
template<class InputIterator, class Size, class NoThrowForwardIterator>
  pair<InputIterator, NoThrowForwardIterator>
    uninitialized_move_n(InputIterator first, Size n, NoThrowForwardIterator result);
```

-6- *Preconditions*: `result + [0, n)` does not overlap with `first + [0, n)`.

-7- *Effects*: Equivalent to:

```
for (; n > 0; ++result,(void) ++first, --n)
  ::new (voidify(*result))
    typename iterator_traits<NoThrowForwardIterator>::value_type(std::move(*deref-move(first));
return {first, result};
```

---

## 4014. LWG 3809 changes behavior of some existing `std::subtract_with_carry_engine` code

**Section:** 29.5.4.4 [rand.eng.sub] **Status:** Ready **Submitter:** Matt Stephanson **Opened:** 2023-11-15 **Last modified:** 2024-10-09

**Priority:** 2

**View all other** issues in [rand.eng.sub].

**Discussion:**

Issue 3809(i) pointed out that `subtract_with_carry_engine<T>` can be seeded with values from a `linear_congruential_engine<T, 40014u, 0u, 2147483563u>` object, which results in narrowing when `T` is less than 32 bits. Part of the resolution was to modify the LCG seed sequence as follows:

```
explicit subtract_with_carry_engine(result_type value);
```

-7- *Effects*: Sets the values of $X_{-r}$, … , $X_{-1}$, in that order, as specified below. If $X_{-1}$ is then 0, sets $c$ to 1; otherwise sets $c$ to 0.

To set the values $X_k$, first construct `e`, a `linear_congruential_engine` object, as if by the following definition:

```
linear_congruential_engine<result_typeuint_least32_t,
                           40014u,0u,2147483563u> e(value == 0u ? default_seed : value);
```

Then, to set each $X_k$, obtain new values $z_0$, … , $z_{n-1}$ from $n = \lceil w/32 \rceil$ successive invocations of `e`. Set $X_k$ to $(\sum_{j=0}^{n-1} z_j \cdot 2^{32j}) \bmod m$.

Inside `linear_congruential_engine`, the seed is reduced modulo 2147483563, so `uint_least32_t` is fine from that point on. This resolution, however, forces `value`, the user-provided seed, to be truncated from `result_type` to `uint_least32_t` before the reduction, which generally will change the result. It also breaks the existing behavior that two seeds are equivalent if they're in the same congruence class modulo the divisor.

*[2024-01-11; Reflector poll]*

Set priority to 2 after reflector poll.

*[2024-01-11; Jonathan comments]*

More precisely, the resolution forces `value` to be *converted* to `uint_least32_t`, which doesn't necessarily truncate, and if it does truncate, it doesn't necessarily change the value. But it will truncate whenever `value_type` is wider than `uint_least32_t`, e.g. for 32-bit `uint_least32_t` you get a different result for `std::ranlux48_base(UINT_MAX + 1LL)()`. The new proposed resolution below restores the old behaviour for that type.

*[2024-10-09; LWG telecon: Move to Ready]*

**Proposed resolution:**

This wording is relative to N4964 *after the wording changes applied* by LWG 3809(i), which had been accepted into the working paper during the Kona 2023-11 meeting.

1. Modify 29.5.4.4 [rand.eng.sub] as indicated:

```
explicit subtract_with_carry_engine(result_type value);
```

-7- *Effects*: Sets the values of $X_{-r}$, … , $X_{-1}$, in that order, as specified below. If $X_{-1}$ is then 0, sets $c$ to 1; otherwise sets $c$ to 0.

To set the values $X_k$, first construct `e`, a `linear_congruential_engine` object, as if by the following definition:

```
linear_congruential_engine<uint_least32_t,
                           40014u,0u,2147483563u> e(value == 0u ? default_seed :
                           static_cast<uint_least32_t>(value % 2147483563u));
```

Then, to set each $X_k$, obtain new values $z_0$, … , $z_{n-1}$ from $n = \lceil w/32 \rceil$ successive invocations of `e`. Set $X_k$ to $(\sum_{j=0}^{n-1} z_j \cdot 2^{32j}) \bmod m$.

---

## 4024. Underspecified destruction of objects created in `std::make_shared_for_overwrite/std::allocate_shared_for_overwrite`

**Section:** 20.3.2.2.7 [util.smartptr.shared.create] **Status:** Ready **Submitter:** Jiang An **Opened:** 2023-12-16 **Last modified:** 2024-08-21

**Priority:** 2

**View other** active issues in [util.smartptr.shared.create].

**View all other** issues in [util.smartptr.shared.create].

**Discussion:**

Currently, only destructions of non-array (sub)objects created in `std::make_shared` and `std::allocate_shared` are specified in 20.3.2.2.7 [util.smartptr.shared.create]. Presumably, objects created in `std::make_shared_for_overwrite` and `std::allocate_shared_for_overwrite` should be destroyed by plain destructor calls.

*[2024-03-11; Reflector poll]*

Set priority to 2 after reflector poll in December 2023.

This was the P1020R1 author's intent (see LWG reflector mail in November 2018) but it was never clarified in the wording. This fixes that.

*[2024-08-21; Move to Ready at LWG telecon]*

**Proposed resolution:**

This wording is relative to N4964.

1. Modify 20.3.2.2.7 [util.smartptr.shared.create] as indicated:

```
template<class T, ...>
  shared_ptr<T> make_shared(args);
template<class T, class A, ...>
  shared_ptr<T> allocate_shared(const A& a, args);
template<class T, ...>
  shared_ptr<T> make_shared_for_overwrite(args);
template<class T, class A, ...>
  shared_ptr<T> allocate_shared_for_overwrite(const A& a, args);
```

[…]

-7- *Remarks:*

[…]

(7.11) — When a (sub)object of non-array type `U` that was initialized by `make_shared`, `make_shared_for_overwrite`, or `allocate_shared_for_overwrite` is to be destroyed, it is destroyed via the expression `pv->~U()` where `pv` points to that object of type `U`.

[…]

---

## 4027. *`possibly-const-range`* should prefer returning `const R&`

**Section:** 25.2 [ranges.syn] **Status:** Ready **Submitter:** Hewill Kang **Opened:** 2023-12-17 **Last modified:** 2024-06-28

**Priority:** 2

**View other** active issues in [ranges.syn].

**View all other** issues in [ranges.syn].

**Discussion:**

*possibly-const-range* currently only returns `const R&` when `R` does not satisfy `constant_range` and `const R` satisfies `constant_range`.

Although it's not clear why we need the former condition, this does diverge from the legacy `std::cbegin` (demo):

```
#include <ranges>

int main() {
  auto r = std::views::single(0)
         | std::views::transform([](int) { return 0; });
  using C1 = decltype(std::ranges::cbegin(r));
  using C2 = decltype(std::cbegin(r));
  static_assert(std::same_as<C1, C2>); // failed
}
```

Since `R` itself is `constant_range`, so *possibly-const-range*, above just returns `R&` and `C1` is `transform_view::iterator<false>`; `std::cbegin` specifies to return `as_const(r).begin()`, which makes that `C2` is `transform_view::iterator<true>` which is different from `C1`.

I believe `const R&` should always be returned if it's a range, regardless of whether `const R` or `R` is a `constant_range`, just as *fmt-maybe-const* in format ranges always prefers `const R` over `R`.

Although it is theoretically possible for `R` to satisfy `constant_range` and that `const R` is a mutable range, such nonsense range type should not be of interest.

This relaxation of constraints allows for maximum consistency with `std::cbegin`, and in some cases can preserve constness to the greatest extent (demo):

```
#include <ranges>

int main() {
  auto r = std::views::single(0) | std::views::lazy_split(0);
  (*std::ranges::cbegin(r)).front() = 42; // ok
  (*std::cbegin(r)).front() = 42; // not ok
}
```

Above, `*std::ranges::cbegin` returns a range of type `const lazy_split_view::outer-iterator<false>::value_type`, which does not satisfy `constant_range` because its reference type is `int&`.

However, `*std::cbegin(r)` returns `lazy_split_view::outer-iterator<true>::value_type` whose reference type is `const int&` and satisfies `constant_range`.

*[2024-03-11; Reflector poll]*

Set priority to 2 after reflector poll. Send to SG9.

*[St. Louis 2024-06-28; LWG and SG9 joint session: move to Ready]*

**Proposed resolution:**

This wording is relative to N4971.

1. Modify 25.2 [ranges.syn], header `<ranges>` synopsis, as indicated:

```
#include <compare>              // see 17.11.1 [compare.syn]
#include <initializer_list>     // see 17.10.2 [initializer.list.syn]
#include <iterator>             // see 24.2 [iterator.synopsis]

namespace std::ranges {
  […]

  // 25.7.22 [range.as.const], as const view
  template<input_range R>
    constexpr auto& possibly-const-range(R& r) noexcept { // exposition only
      if constexpr (input~~constant~~_range<const R>~~ && !constant_range<R>~~) {
        return const_cast<const R&>(r);
      } else {
        return r;
      }
    }

  […]
}
```

---

## 4044. Confusing requirements for `std::print` on POSIX platforms

**Section:** 31.7.10 [print.fun] **Status:** Ready **Submitter:** Jonathan Wakely **Opened:** 2024-01-24 **Last modified:** 2024-06-24

**Priority:** 3

**View other** active issues in [print.fun].

**View all other** issues in [print.fun].

**Discussion:**

The effects for `vprintf_unicode` say:

If `stream` refers to a terminal capable of displaying Unicode, writes `out` to the terminal using the native Unicode API; if `out` contains invalid code units, the behavior is undefined and implementations are encouraged to diagnose it. Otherwise writes `out` to `stream` unchanged. If the native Unicode API is used, the function flushes `stream` before writing `out`.

[*Note 1*: On POSIX and Windows, `stream` referring to a terminal means that, respectively, `isatty(fileno(stream))` and `GetConsoleMode(_get_osfhandle(_fileno(stream)), ...)` return nonzero. — *end note*]

[*Note 2*: On Windows, the native Unicode API is `WriteConsoleW`. — *end note*]

-8- *Throws*: [...]

-9- *Recommended practice*: If invoking the native Unicode API requires transcoding, implementations should substitute invalid code units with U+FFFD REPLACEMENT CHARACTER per the Unicode Standard, Chapter 3.9 U+FFFD Substitution in Conversion.

The very explicit mention of `isatty` for POSIX platforms has confused at least two implementers into thinking that we're supposed to use `isatty`, and supposed to do something differently based on what it returns. That seems consistent with the nearly identical wording in 28.5.2.2 [format.string.std] paragraph 12, which says "Implementations should use either UTF-8, UTF-16, or UTF-32, on platforms capable of displaying Unicode text in a terminal" and then has a note explicitly saying this is the case for Windows-based and many POSIX-based operating systems. So it seems clear that POSIX platforms are supposed to be considered to have "a terminal capable of displaying Unicode text", and so `std::print` should use `isatty` and then use a native Unicode API, and diagnose invalid code units.

This is a problem however, because `isatty` needs to make a system call on Linux, adding 500ns to every `std::print` call. This results in a 10x slowdown on Linux, where `std::print` can take just 60ns without the `isatty` check.

From discussions with Tom Honermann I learned that the "native Unicode API" wording is only relevant on Windows. This makes sense, because for POSIX platforms, writing to a terminal is done using the usual stdio functions, so there's no need to treat a terminal differently to any other file stream. And substitution of invalid code units with U+FFFD is recommended for Windows because that's what typical modern terminals do on POSIX platforms, so requiring the implementation to do that on Windows gives consistent behaviour. But the implementation doesn't need to do anything to make that happen with a POSIX terminal, it happens anyway. So the `isatty` check is unnecessary for POSIX platforms, and the note mentioning it just causes confusion and has no benefit.

Secondly, there initially seems to be a contradiction between the "implementations are encouraged to diagnose it" wording and the later *Recommended practice*. In fact, there's no contradiction because the native Unicode API might accept UTF-8 and therefore require no transcoding, and so the *Recommended practice* wouldn't apply. The intention is that diagnosing invalid UTF-8 is still desirable in this case, but how should it be diagnosed? By writing an error to the terminal alongside the formatted string? Or by substituting U+FFFD maybe? If the latter is the intention, why is one suggestion in the middle of the *Effects*, and one given as *Recommended practice*?

The proposed resolution attempts to clarify that a "native Unicode API" is only needed if that's how you display Unicode on the terminal. It also moves the flushing requirement to be adjacent to the other requirements for systems using a native Unicode API instead of on its own later in the paragraph. And the suggestion to diagnose invalid code units is moved into the *Recommended practice* and clarified that it's only relevant if using a native Unicode API. I'm still not entirely happy with encouragement to diagnose invalid code units without giving any clue as to how that should be done. What does it mean to diagnose something at runtime? That's novel for the C++ standard. The way it's currently phrased seems to imply something other than U+FFFD substitution should be done, although that seems the most obvious implementation to me.

*[2024-03-12; Reflector poll]*

Set priority to 3 after reflector poll and send to SG16.

**Previous resolution [SUPERSEDED]:**

This wording is relative to N4971.

1. Modify 31.7.6.3.5 [ostream.formatted.print] as indicated:

```
void vprint_unicode(ostream& os, string_view fmt, format_args args);
void vprint_nonunicode(ostream& os, string_view fmt, format_args args);
```

-3- *Effects*: Behaves as a formatted output function (31.7.6.3.1 [ostream.formatted.reqmts]) of `os`, except that:

(3.1) – failure to generate output is reported as specified below, and
(3.2) – any exception thrown by the call to `vformat` is propagated without regard to the value of `os.exceptions()` and without turning on `ios_base::badbit` in the error state of `os`.

After constructing a `sentry` object, the function initializes an automatic variable via

```
string out = vformat(os.getloc(), fmt, args);
```

If the function is `vprint_unicode` and `os` is a stream that refers to a terminal capable of displaying Unicode via a native Unicode API, which is determined in an implementation-defined manner, flushes `os` and then writes `out` to the terminal using the native Unicode API; if `out` contains invalid code units, the behavior is undefined ~~and implementations are encouraged to diagnose it~~. ~~If the native Unicode API is used, the function flushes `os` before writing `out`.~~ Otherwise, (if `os` is not such a stream or the function is `vprint_nonunicode`), inserts the character sequence [`out.begin()`,`out.end()`) into `os`. If writing to the terminal or inserting into `os` fails, calls `os.setstate(ios_base::badbit)` (which may throw `ios_base::failure`).

-4- *Recommended practice*: For `vprint_unicode`, if invoking the native Unicode API requires transcoding, implementations should substitute invalid code units with U+FFFD REPLACEMENT CHARACTER per the Unicode Standard, Chapter 3.9 U+FFFD Substitution in Conversion. If invoking the native Unicode API does not require transcoding, implementations are encouraged to diagnose invalid code units.

2. Modify 31.7.10 [print.fun] as indicated:

```
void vprint_unicode(FILE* stream, string_view fmt, format_args args);
```

-6- *Preconditions*: `stream` is a valid pointer to an output C stream.

-7- *Effects*: The function initializes an automatic variable via

```
string out = vformat(fmt, args);
```

If `stream` refers to a terminal capable of displaying Unicode via a native Unicode API, flushes `stream` and then writes `out` to the terminal using the native Unicode API; if `out` contains invalid code units, the behavior is undefined ~~and implementations are encouraged to diagnose it~~. Otherwise writes `out` to `stream` unchanged. ~~If the native Unicode API is used, the function flushes `stream` before writing `out`.~~

[*Note 1*: On ~~POSIX and~~ Windows~~,~~ the native Unicode API is `WriteConsoleW` and `stream` referring to a terminal means that~~,~~ ~~respectively, `isatty(fileno(stream))` and~~ `GetConsoleMode(_get_osfhandle(_fileno(stream)), ...)` return nonzero. — *end note*]

~~[*Note 2*: On Windows, the native Unicode API is `WriteConsoleW`. — *end note*]~~

-8- *Throws*: [...]

-9- *Recommended practice*: If invoking the native Unicode API requires transcoding, implementations should substitute invalid code units with U+FFFD REPLACEMENT CHARACTER per the Unicode Standard, Chapter 3.9 U+FFFD Substitution in Conversion. If invoking the native Unicode API does not require transcoding, implementations are encouraged to diagnose invalid code units.

*[2024-03-12; Jonathan updates wording based on SG16 feedback]*

SG16 reviewed the issue and approved the proposed resolution with the wording about diagnosing invalid code units removed.

SG16 favors removing the following text (both occurrences) from the proposed wording. This is motivated by a lack of understanding regarding what it means to diagnose such invalid code unit sequences given that the input is likely provided at run-time.

> If invoking the native Unicode API does not require transcoding, implementations are encouraged to diagnose invalid code units.

Some concern was expressed regarding how the current wording is structured. At present, the wording leads with a Windows centric perspective; if the stream refers to a terminal ... use the native Unicode API ... otherwise write code units to the stream. It might be an improvement to structure the wording such that use of the native Unicode API is presented as a fallback for implementations that require its use when writing directly to the stream is not sufficient to produce desired results. In other words, the wording should permit direct writing to the stream even when the stream is directed to a terminal and a native Unicode API is available when the implementation has reason to believe that doing so will produce the correct results. For example, Microsoft's HoloLens has a Windows based operating system, but it only supports use of UTF-8 as the system code page and therefore would not require the native Unicode API bypass; implementations for it could avoid the overhead of checking to see if the stream is directed to a console.

**Previous resolution [SUPERSEDED]:**

This wording is relative to N4971.

1. Modify 31.7.6.3.5 [ostream.formatted.print] as indicated:

```
void vprint_unicode(ostream& os, string_view fmt, format_args args);
void vprint_nonunicode(ostream& os, string_view fmt, format_args args);
```

-3- *Effects*: Behaves as a formatted output function (31.7.6.3.1 [ostream.formatted.reqmts]) of `os`, except that:

(3.1) – failure to generate output is reported as specified below, and
(3.2) – any exception thrown by the call to `vformat` is propagated without regard to the value of `os.exceptions()` and without turning on `ios_base::badbit` in the error state of `os`.

After constructing a `sentry` object, the function initializes an automatic variable via

```
string out = vformat(os.getloc(), fmt, args);
```

If the function is `vprint_unicode` and `os` is a stream that refers to a terminal that is only capable of displaying Unicode via a native Unicode API, which is determined in an implementation-defined manner, flushes `os` and then writes `out` to the terminal using the native Unicode API; if `out` contains invalid code units, the behavior is undefined ~~and implementations are encouraged to diagnose it~~. ~~If the native Unicode API is used, the function flushes `os` before writing `out`.~~ Otherwise, (if `os` is not such a stream or the function is `vprint_nonunicode`), inserts the character sequence `[out.begin(),out.end())` into `os`. If writing to the terminal or inserting into `os` fails, calls `os.setstate(ios_base::badbit)` (which may throw `ios_base::failure`).

-4- *Recommended practice*: For `vprint_unicode`, if invoking the native Unicode API requires transcoding, implementations should substitute invalid code units with U+FFFD REPLACEMENT CHARACTER per the Unicode Standard, Chapter 3.9 U+FFFD Substitution in Conversion.

2. Modify 31.7.10 [print.fun] as indicated:

```
void vprint_unicode(FILE* stream, string_view fmt, format_args args);
```

-6- *Preconditions*: `stream` is a valid pointer to an output C stream.

-7- *Effects*: The function initializes an automatic variable via

```
string out = vformat(fmt, args);
```

If `stream` refers to a terminal that is only capable of displaying Unicode via a native Unicode API, flushes `stream` and then writes `out` to the terminal using the native Unicode API; if `out` contains invalid code units, the behavior is undefined ~~and implementations are encouraged to diagnose it~~. Otherwise writes `out` to `stream` unchanged. ~~If the native Unicode API is used, the function flushes `stream` before writing `out`.~~

[*Note 1*: On ~~POSIX and~~ Windows~~,~~ the native Unicode API is `WriteConsoleW` and `stream` referring to a terminal means that~~, respectively, `isatty(fileno(stream))` and~~ `GetConsoleMode(_get_osfhandle(_fileno(stream)), ...)` return nonzero. — *end note*]

~~[*Note 2*: On Windows, the native Unicode API is `WriteConsoleW`. — *end note*]~~

-8- *Throws*: [...]

-9- *Recommended practice*: If invoking the native Unicode API requires transcoding, implementations should substitute invalid code units with U+FFFD REPLACEMENT CHARACTER per the Unicode Standard, Chapter 3.9 U+FFFD Substitution in Conversion.

*[2024-03-19; Tokyo: Jonathan updates wording after LWG review]*

Split the *Effects*: into separate bullets for the "native Unicode API" and "otherwise" cases. Remove the now-redundant "if `os` is not such a stream" parenthesis.

*[St. Louis 2024-06-24; move to Ready.]*

**Proposed resolution:**

This wording is relative to N4971.

1. Modify 31.7.6.3.5 [ostream.formatted.print] as indicated:

```
void vprint_unicode(ostream& os, string_view fmt, format_args args);
void vprint_nonunicode(ostream& os, string_view fmt, format_args args);
```

-3- *Effects*: Behaves as a formatted output function (31.7.6.3.1 [ostream.formatted.reqmts]) of `os`, except that:

(3.1) – failure to generate output is reported as specified below, and
(3.2) – any exception thrown by the call to `vformat` is propagated without regard to the value of `os.exceptions()` and without turning on `ios_base::badbit` in the error state of `os`.

-?- After constructing a `sentry` object, the function initializes an automatic variable via

```
string out = vformat(os.getloc(), fmt, args);
```

(?.1) – If the function is `vprint_unicode` and `os` is a stream that refers to a terminal that is only capable of displaying Unicode via a native Unicode API, which is determined in an implementation-defined manner, flushes `os` and then writes `out` to the terminal using the native Unicode API; if `out` contains invalid code units, the behavior is undefined ~~and implementations are encouraged to diagnose it. If the native Unicode API is used, the function flushes `os` before writing `out`~~.
(?.2) – Otherwise, ~~(if `os` is not such a stream or the function is `vprint_nonunicode`),~~ inserts the character sequence [`out.begin()`,`out.end()`) into `os`.

-?- If writing to the terminal or inserting into `os` fails, calls `os.setstate(ios_base::badbit)` (which may throw `ios_base::failure`).

-4- *Recommended practice*: For `vprint_unicode`, if invoking the native Unicode API requires transcoding, implementations should substitute invalid code units with U+FFFD REPLACEMENT CHARACTER per the Unicode Standard, Chapter 3.9 U+FFFD Substitution in Conversion.

2. Modify 31.7.10 [print.fun] as indicated:

```
void vprint_unicode(FILE* stream, string_view fmt, format_args args);
```

-6- *Preconditions*: `stream` is a valid pointer to an output C stream.

-7- *Effects*: The function initializes an automatic variable via

```
string out = vformat(fmt, args);
```

(7.1) – If `stream` refers to a terminal that is only capable of displaying Unicode via a native Unicode API, flushes `stream` and then writes `out` to the terminal using the native Unicode API; if `out` contains invalid code units, the behavior is undefined ~~and implementations are encouraged to diagnose it~~.
(7.2) – Otherwise writes `out` to `stream` unchanged.

~~If the native Unicode API is used, the function flushes `stream` before writing `out`.~~

[*Note 1*: On ~~POSIX and~~ Windows~~,~~ the native Unicode API is `WriteConsoleW` and `stream` referring to a terminal means that~~, respectively, `isatty(fileno(stream))` and~~ `GetConsoleMode(_get_osfhandle(_fileno(stream)), ...)` return~~s~~ nonzero. — *end note*]

~~[*Note 2*: On Windows, the native Unicode API is `WriteConsoleW`. — *end note*]~~

-8- *Throws*: [...]

-9- *Recommended practice*: If invoking the native Unicode API requires transcoding, implementations should substitute invalid code units with U+FFFD REPLACEMENT CHARACTER per the Unicode Standard, Chapter 3.9 U+FFFD Substitution in Conversion.

## 4064. Clarify that `std::launder` is not needed when using the result of `std::memcpy`

**Section:** 27.5.1 [cstring.syn] **Status:** Ready **Submitter:** Jan Schultke **Opened:** 2024-04-05 **Last modified:** 2024-06-28

**Priority:** 3

**Discussion:**

```
int x = 0;
alignas(int) std::byte y[sizeof(int)];
int z = *static_cast<int*>(std::memcpy(y, &x, sizeof(int)));
```

This example should be well-defined, even without the use of `std::launder`. `std::memcpy` implicitly creates an `int` inside `y`, and https://www.iso-9899.info/n3047.html#7.26.2.1p3 states that

> The `memcpy` function returns the value of [the destination operand].

In conjunction with 27.5.1 [cstring.syn] p3, this presumably means that `std::memcpy` returns a pointer to the (first) implicitly-created object, and no use of `std::launder` is necessary.

The wording should be clarified to clearly support this interpretation or reject it.

*[2024-06-24; Reflector poll]*

Set priority to 3 after reflector poll.

**Previous resolution [SUPERSEDED]:**

This wording is relative to N4971.

1. Modify 27.5.1 [cstring.syn] as indicated:

> -3- The functions `memcpy` and `memmove` are signal-safe (17.13.5 [support.signal]). Both functions implicitly create objects (6.7.2 [intro.object]) in the destination region of storage immediately prior to copying the sequence of characters to the destination. Both functions return a pointer to a suitable created object.

*[St. Louis 2024-06-26; CWG suggested improved wording]*

*[St. Louis 2024-06-28; LWG: move to Ready]*

**Proposed resolution:**

This wording is relative to N4981.

1. Modify 27.5.1 [cstring.syn] as indicated:

> -3- The functions `memcpy` and `memmove` are signal-safe (17.13.5 [support.signal]). ~~Both~~ Each of these functions implicitly ~~create~~ creates objects (6.7.2 [intro.object]) in the destination region of storage immediately prior to copying the sequence of characters to the destination. Each of these functions returns a pointer to a suitable created object, if any, otherwise the value of the first parameter.

---

## 4072. `std::optional` comparisons: constrain harder

**Section:** 22.5.8 [optional.comp.with.t] **Status:** Ready **Submitter:** Jonathan Wakely **Opened:** 2024-04-19 **Last modified:** 2024-08-21

**Priority:** 1

**View all other** issues in [optional.comp.with.t].

**Discussion:**

P2944R3 added constraints to `std::optional`'s comparisons, e.g.

```
template<class T, class U> constexpr bool operator==(const optional<T>& x, const optional<U>& y);
```

-1- ~~*Mandates*~~*Constraints*: The expression `*x == *y` is well-formed and its result is convertible to `bool`.

…

```
template<class T, class U> constexpr bool operator==(const optional<T>& x, const U& v);
```

-1- ~~*Mandates*~~*Constraints*: The expression `*x == v` is well-formed and its result is convertible to `bool`.

But I don't think the constraint on the second one (the "compare with value") is correct. If we try to compare two optionals that can't be compared, such as `optional<void*>` and `optional<int>`, then the first overload is not valid due to the new constraints, and so does not participate in overload resolution. But that means we now consider the second overload, but that's ambiguous. We could either use `operator==<void*, optional<int>>` or we could use `operator==<optional<void*>, int>` with the arguments reversed (using the C++20 default comparison rules). We never even get as far as checking the new constraints on those overloads, because they're simply ambiguous.

Before P2944R3 overload resolution always would have selected the first overload, for comparing two optionals. But because that is now constrained away, we consider an overload that should never be used for comparing two optionals. The solution is to add an additional constraint to the "compare with value" overloads so that they won't be used when the "value" is really another optional.

A similar change was made to `optional`'s `operator<=>` by LWG 3566(i), and modified by LWG 3746(i). I haven't analyzed whether we need the modification here too.

The proposed resolution (without *`is-derived-from-optional`*) has been implemented and tested in libstdc++.

*[2024-06-24; Reflector poll]*

Set priority to 1 after reflector poll. LWG 3746(i) changes might be needed here too, i.e, consider types derived from `optional`, not only `optional` itself.

*[2024-08-21; Move to Ready at LWG telecon]*

**Proposed resolution:**

This wording is relative to N4981.

1. Modify 22.5.8 [optional.comp.with.t] as indicated:

```
template<class T, class U> constexpr bool operator==(const optional<T>& x, const U& v);
```

-1- *Constraints*: `U` is not a specialization of `optional`. The expression `*x == v` is well-formed and its result is convertible to `bool`.

```
template<class T, class U> constexpr bool operator==(const T& v, const optional<U>& x);
```

-3- *Constraints*: `T` is not a specialization of `optional`. The expression `v == *x` is well-formed and its result is convertible to `bool`.

```
template<class T, class U> constexpr bool operator!=(const optional<T>& x, const U& v);
```

-5- *Constraints*: `U` is not a specialization of `optional`. The expression `*x != v` is well-formed and its result is convertible to `bool`.

```
template<class T, class U> constexpr bool operator!=(const T& v, const optional<U>& x);
```

-7- *Constraints*: `T` is not a specialization of `optional`. The expression `v != *x` is well-formed and its result is convertible to `bool`.

```
template<class T, class U> constexpr bool operator<(const optional<T>& x, const U& v);
```

-9- *Constraints*: `U` is not a specialization of `optional`. The expression `*x < v` is well-formed and its result is convertible to `bool`.

```
template<class T, class U> constexpr bool operator<(const T& v, const optional<U>& x);
```

-11- *Constraints*: `T` is not a specialization of `optional`. The expression `v < *x` is well-formed and its result is convertible to `bool`.

```
template<class T, class U> constexpr bool operator>(const optional<T>& x, const U& v);
```

-13- *Constraints*: `U` is not a specialization of `optional`. The expression `*x > v` is well-formed and its result is convertible to `bool`.

```
template<class T, class U> constexpr bool operator>(const T& v, const optional<U>& x);
```

-15- *Constraints*: `T` is not a specialization of `optional`. The expression `v > *x` is well-formed and its result is convertible to `bool`.

```
template<class T, class U> constexpr bool operator<=(const optional<T>& x, const U& v);
```

-17- *Constraints*: `U` is not a specialization of `optional`. The expression `*x <= v` is well-formed and its result is convertible to `bool`.

```
template<class T, class U> constexpr bool operator<=(const T& v, const optional<U>& x);
```

-19- *Constraints*: `T` is not a specialization of `optional`. The expression `v <= *x` is well-formed and its result is convertible to `bool`.

```
template<class T, class U> constexpr bool operator>=(const optional<T>& x, const U& v);
```

-21- *Constraints*: `U` is not a specialization of `optional`. The expression `*x >= v` is well-formed and its result is convertible to `bool`.

```
template<class T, class U> constexpr bool operator>=(const T& v, const optional<U>& x);
```

-23- *Constraints*: `T` is not a specialization of `optional`. The expression `v >= *x` is well-formed and its result is convertible to `bool`.

---

## 4085. `ranges::generate_random`'s helper lambda should specify the return type

**Section:** 26.12.2 [alg.rand.generate] **Status:** Ready **Submitter:** Hewill Kang **Opened:** 2024-04-28 **Last modified:** 2024-10-09

**Priority:** 2

**View other** active issues in [alg.rand.generate].

**View all other** issues in [alg.rand.generate].

**Discussion:**

The non-specialized case of `generate_random(r, g, d)` would call `ranges::generate(r, [&d, &g] { return invoke(d, g); })`. However, the lambda does not explicitly specify the return type, which leads to a hard error when `invoke` returns a reference to the object that is not copyable or `R` is not the `output_range` for `decay_t<invoke_result_t<D&, G&>>`.

**Previous resolution [SUPERSEDED]:**

This wording is relative to N4981.

1. Modify 26.12.2 [alg.rand.generate] as indicated:

```
template<class R, class G, class D>
  requires output_range<R, invoke_result_t<D&, G&>> && invocable<D&, G&> &&
           uniform_random_bit_generator<remove_cvref_t<G>>
constexpr borrowed_iterator_t<R> ranges::generate_random(R&& r, G&& g, D&& d);
```

-5- *Effects*:

(5.1) — […]

(5.2) — […]

(5.3) — Otherwise, calls:

```
ranges::generate(std::forward<R>(r), [&d, &g] -> decltype(auto) { return invoke(d, g); });
```

-6- *Returns*: `ranges::end(r)`

-7- *Remarks*: The effects of `generate_random(r, g, d)` shall be equivalent to

```
ranges::generate(std::forward<R>(r), [&d, &g] -> decltype(auto) { return invoke(d, g); })
```

*[2024-05-12, Hewill Kang provides improved wording]*

*[2024-08-02; Reflector poll]*

Set priority to 2 after reflector poll.

*[2024-10-09; LWG telecon: Move to Ready]*

**Proposed resolution:**

This wording is relative to N4981.

1. Modify 29.5.2 [rand.synopsis], header <random> synopsis, as indicated:

```
#include <initializer_list>     // see 17.10.2 [initializer.list.syn]

namespace std {
  […]
  namespace ranges {
    […]
    template<class R, class G, class D>
      requires output_range<R, invoke_result_t<D&, G&>> && invocable<D&, G&> &&
               uniform_random_bit_generator<remove_cvref_t<G>> &&
               is_arithmetic_v<invoke_result_t<D&, G&>>
    constexpr borrowed_iterator_t<R> generate_random(R&& r, G&& g, D&& d);

    template<class G, class D, output_iterator<invoke_result_t<D&, G&>> O, sentinel_for<O> S>
      requires invocable<D&, G&> && uniform_random_bit_generator<remove_cvref_t<G>> &&
               is_arithmetic_v<invoke_result_t<D&, G&>>
    constexpr O generate_random(O first, S last, G&& g, D&& d);
  }
  […]
}
```

2. Modify 26.12.2 [alg.rand.generate] as indicated:

```
template<class R, class G, class D>
  requires output_range<R, invoke_result_t<D&, G&>> && invocable<D&, G&> &&
           uniform_random_bit_generator<remove_cvref_t<G>> &&
           is_arithmetic_v<invoke_result_t<D&, G&>>
constexpr borrowed_iterator_t<R> ranges::generate_random(R&& r, G&& g, D&& d);
```

-5- *Effects*:

[…]

```
template<class G, class D, output_iterator<invoke_result_t<D&, G&>> O, sentinel_for<O> S>
  requires invocable<D&, G&> && uniform_random_bit_generator<remove_cvref_t<G>> &&
           is_arithmetic_v<invoke_result_t<D&, G&>>
constexpr O ranges::generate_random(O first, S last, G&& g, D&& d);
```

-8- *Effects*: Equivalent to:

[…]

---

## 4112. *has-arrow* should required `operator->()` to be `const`-qualified

**Section:** 25.5.2 [range.utility.helpers] **Status:** Ready **Submitter:** Hewill Kang **Opened:** 2024-06-22 **Last modified:** 2024-06-24

**Priority:** Not Prioritized

**Discussion:**

The helper concept *has-arrow* in 25.5.2 [range.utility.helpers] does not require that `I::operator->()` be `const`-qualified, which is inconsistent with the constraints on `reverse_iterator` and `common_iterator`'s `operator->()` as the latter two both require the underlying iterator has `const operator->()` member.

We should enhance the semantics of *has-arrow* so that *implicit expression variations* (18.2 [concepts.equality]) prohibit silly games.

*[St. Louis 2024-06-24; move to Ready]*

**Proposed resolution:**

This wording is relative to N4981.

1. Modify 25.5.2 [range.utility.helpers] as indicated:

[…]

```
template<class I>
  concept has-arrow =                                       // exposition only
    input_iterator<I> && (is_pointer_v<I> || requires(const I i) { i.operator->(); });
```

[…]

---

## 4134. Issue with Philox algorithm specification

**Section:** 29.5.4.5 [rand.eng.philox] **Status:** Ready **Submitter:** Ilya A. Burylov **Opened:** 2024-08-06 **Last modified:** 2024-08-21

**Priority:** 1

**View other** active issues in [rand.eng.philox].

**View all other** issues in [rand.eng.philox].

**Discussion:**

The P2075R6 proposal introduced the Philox engine and described the algorithm closely following the original paper (further Random123sc11).

Matt Stephanson implemented P2075R6 and the 10000'th number did not match. Further investigation revealed several places in Random123sc11 algorithm description, which either deviate from the reference implementation written by Random123sc11 authors or loosely defined, which opens the way for different interpretations.

All major implementations of the Philox algorithm (NumPy, Intel MKL, Nvidia cuRAND, etc.) match the reference implementation written by Random123sc11 authors and we propose to align wording with that.

The rationale of proposed changes:

1. Random123sc11 refers to the permutation step as "the inputs are permuted using the Threefish N-word P-box", thus the P2075R6 permutation table ([tab:rand.eng.philox.f]) is taken from Threefish algorithm. But the permutation for N=4 in this table does not match the reference implementations. It's worth noting that while Random123sc11 described the Philox algorithm for N=8 and N=16, there are no known reference implementations of it either provided by authors or implemented by other libraries. We proposed to drop N=8 and N=16 for now and update the permutation indices for N=4 to match the reference implementation. Note: the proposed resolution allows extending N > 4 cases in the future.

2. The original paper describes the S-box update for X values in terms of `L'` and `R'` but does not clarify their ordering as part of the counter. In order to match Random123sc11 reference implementation the ordering should be swapped.

3. Philox alias templates should be updated, because the current ordering of constants matches the specific optimization in the reference Random123sc11 implementation and not the generic algorithm description.

All proposed modifications below are confirmed by:

- Philox algorithm coauthor Mark Moraes who is planning to publish errata for the original Random123sc11 Philox paper.
- Matt Stephanson, who originally found the mismatch in P2075R6
- The updated reference implementation.

*[2024-08-21; Reflector poll]*

Set priority to 1 after reflector poll.

*[2024-08-21; Move to Ready at LWG telecon]*

**Proposed resolution:**

This wording is relative to N4988.

1. Modify 29.5.4.5 [rand.eng.philox], [tab:rand.eng.philox.f] as indicated (This effectively reduces 16 data columns to 4 data columns and 4 data rows to 2 data rows):

Table 101 — Values for the word permutation $f_n(j)$ [tab:rand.eng.philox.f]

| $f_n(j)$ | | $j$ | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9** | **10** | **11** | **12** | **13** | **14** | **15** |
| $n$ | **2** | 0 | 1 | | | | | | | | | | | | | | |
| | **4** | 2 | 1 | 0 | 3 | | | | | | | | | | | | |
| | **8** | 2 | 1 | 4 | 7 | 6 | 5 | 0 | 3 | | | | | | | | |
| | **16** | 0 | 9 | 2 | 13 | 6 | 11 | 4 | 15 | 10 | 7 | 12 | 3 | 14 | 5 | 8 | 1 |

2. Modify 29.5.4.5 [rand.eng.philox] as indicated:

-4- […]

(4.1) — […]

(4.2) — The following computations are applied to the elements of the $V$ sequence:

$$X_{2k+0} = \text{mulhi}\sout{\text{mullo}}(V_{2k+1}, M_k, w) \text{ xor } key_k^q \text{ xor } V_{2k+1}$$

$$X_{2k+1} = \text{mullo}\sout{\text{mulhi}}(V_{2k+1}, M_k, w) \sout{\text{xor } key_k^q \text{ xor } V_{2k}}$$

-5- […]

-6- *Mandates*:

(6.1) — […]

(6.2) — `n == 2 || n == 4` ~~`|| n == 8 || n == 16`~~ is `true`, and

(6.3) — […]

(6.4) — […]

3. Modify 29.5.6 [rand.predef] as indicated:

```
using philox4x32 =
      philox_engine<uint_fast32_t, 32, 4, 10,
        0xCD9E8D57~~0xD2511F53~~, 0x9E3779B9, 0xD2511F53~~0xCD9E8D57~~, 0xBB67AE85>;
```

-11- *Required behavior*: The 10000th consecutive invocation a default-constructed object of type `philox4x32` produces the value `1955073260`.

```
using philox4x64 =
      philox_engine<uint_fast64_t, 64, 4, 10,
        0xCA5A826395121157~~0xD2E7470EE14C6C93~~, 0x9E3779B97F4A7C15, 0xD2E7470EE14C6C93~~0xCA5A826395121157~~, 0xBB67AE8584CAA73B>;
```

-12- *Required behavior*: The 10000th consecutive invocation a default-constructed object of type `philox4x64` produces the value 3409172418970261260.

---

## 4154. The Mandates for `std::packaged_task`'s constructor from a callable entity should consider decaying

**Section:** 32.10.10.2 [futures.task.members] **Status:** Ready **Submitter:** Jiang An **Opened:** 2024-09-18 **Last modified:** 2024-10-09

**Priority:** 3

**View other** active issues in [futures.task.members].

**View all other** issues in [futures.task.members].

**Discussion:**

Currently, 32.10.10.2 [futures.task.members]/3 states:

> *Mandates*: `is_invocable_r_v<R, F&, ArgTypes...>` is `true`.

where `F&` can be a reference to a cv-qualified function object type.

However, in mainstream implementations (libc++, libstdc++, and MSVC STL), the stored task object always has a cv-unqualified type, and thus the cv-qualification is unrecognizable in `operator()`.

Since 22.10.17.3.2 [func.wrap.func.con] uses a decayed type, perhaps we should also so specify for `std::packaged_task`.

*[2024-10-02; Reflector poll]*

Set priority to 3 after reflector poll.

"Fix preconditions, `f` doesn't need to be invocable, we only invoke the copy."

**Previous resolution [SUPERSEDED]:**

> This wording is relative to N4988.
>
> 1. Modify 32.10.10.2 [futures.task.members] as indicated:
>
> -3- *Mandates*: `is_invocable_r_v<R, ~~F~~decay_t<F>&, ArgTypes...>` is `true`.
>
> [...]
>
> -5- *Effects*: Constructs a new `packaged_task` object with a shared state and initializes the object's stored task of type `decay_t<F>` with `std::forward<F>(f)`.

*[2024-10-02; Jonathan provides improved wording]*

Drop preconditions as suggested on reflector.

*[2024-10-02; LWG telecon]*

Clarify that "of type `decay_t<F>`" is supposed to be specifying the type of the stored task.

*[2024-10-09; LWG telecon: Move to Ready]*

**Proposed resolution:**

This wording is relative to N4988.

1. Modify 32.10.10.2 [futures.task.members] as indicated:

```
template<class F>
  explicit packaged_task(F&& f);
```

-2- *Constraints*: `remove_cvref_t<F>` is not the same type as `packaged_task<R(ArgTypes...)>`.

-3- *Mandates*: `is_invocable_r_v<R, ~~F~~decay_t<F>&, ArgTypes...>` is `true`.

~~-4- *Preconditions*: Invoking a copy of `f` behaves the same as invoking `f`.~~

-5- *Effects*: Constructs a new `packaged_task` object with a stored task of type `decay_t<F>` and a shared state . Initializes ~~and initializes~~ the object's stored task with `std::forward<F>(f)`.

---

## 4164. Missing guarantees for `forward_list` modifiers

**Section:** 23.3.7.5 [forward.list.modifiers] **Status:** Ready **Submitter:** Jonathan Wakely **Opened:** 2024-10-05 **Last modified:** 2024-10-09

**Priority:** 3

**View all other** issues in [forward.list.modifiers].

**Discussion:**

The new `std::list` members added by P1206R7, `insert_range(const_iterator, R&&)`, `prepend_range(R&&)`, and `append_range(R&&)`, have the same exception safety guarantee as `std::list::insert(const_iterator, InputIterator, InputIterator)`, which is:

> *Remarks*: Does not affect the validity of iterators and references. If an exception is thrown, there are no effects.

This guarantee was achieved for the new `list` functions simply by placing them in the same set of declarations as the existing `insert` overloads, at the start of 23.3.9.4 [list.modifiers].

However, the new `std::forward_list` members, `insert_range_after(const_iterator, R&&)` and `prepend_range(R&&)`, do not have the same guarantee as `forward_list::insert_after`. This looks like an omission caused by the fact that `insert_after`'s exception safety guarantee is given in a separate paragraph at the start of 23.3.7.5 [forward.list.modifiers]:

> None of the overloads of `insert_after` shall affect the validity of iterators and references, and `erase_after` shall invalidate only iterators and references to the erased elements. If an exception is thrown during `insert_after` there shall be no effect.

I think we should give similar guarantees for `insert_range_after` and `prepend_range`. The change might also be appropriate for `emplace_after` as well. A "no effects" guarantee is already given for `push_front` and `emplace_front` in 23.2.2.2 [container.reqmts] p66, although that doesn't say anything about iterator

invalidation so we might want to add that to 23.3.7.5 [forward.list.modifiers] too. For the functions that insert a single element, it's trivial to not modify the list if allocating a new node of constructing the element throws. The strong exception safety guarantee for the multi-element insertion functions is easily achieved by inserting into a temporary `forward_list` first, then using `splice_after` which is non-throwing.

*[2024-10-09; Reflector poll]*

Set priority to 3 after reflector poll.

It was suggested to change "If an exception is thrown by any of these member functions that insert elements there is no effect on the forward_list" to simply "If an exception is thrown by any of these member functions there is no effect on the forward_list"

**Previous resolution [SUPERSEDED]:**

This wording is relative to N4988.

1. Change 23.3.7.5 [forward.list.modifiers] as indicated:

> None of the ~~overloads of `insert_after` shall~~ member functions in this subclause that insert elements affect the validity of iterators and references, and `erase_after` ~~shall invalidate~~ invalidates only iterators and references to the erased elements. If an exception is thrown ~~during `insert_after`~~ by any of these member functions there ~~shall be~~ is no effect on the `forward_list`.

*[2024-10-09; LWG suggested improved wording]*

The proposed resolution potentially mandates a change to `resize` when increasing the size, requiring implementations to "roll back" earlier insertions if a later one throws, so that the size is left unchanged. It appears that libstdc++ and MSVC already do this, libc++ does not.

*[2024-10-09; LWG telecon: Move to Ready]*

**Proposed resolution:**

This wording is relative to N4988.

1. Change 23.3.7.5 [forward.list.modifiers] as indicated:

> ~~None of the overloads of `insert_after` shall affect the validity of iterators and references, and `erase_after` shall invalidate only iterators and references to the erased elements.~~ The member functions in this subclause do not affect the validity of iterators and references when inserting elements, and when erasing elements invalidate iterators and references to the erased elements only. If an exception is thrown ~~during `insert_after`~~ by any of these member functions there ~~shall be~~ is no effect on the container.

# Tentatively Ready Issues

---

## 3216. Rebinding the allocator before calling `construct`/`destroy` in `allocate_shared`

**Section:** 20.3.2.2.7 [util.smartptr.shared.create] **Status:** Tentatively Ready **Submitter:** Billy O'Neal III **Opened:** 2019-06-11 **Last modified:** 2024-10-02

**Priority:** 3

**View other** active issues in [util.smartptr.shared.create].

**View all other** issues in [util.smartptr.shared.create].

**Discussion:**

The new `allocate_shared` wording says we need to rebind the allocator back to `T`'s type before we can call `construct` or `destroy`, but this is suboptimal (might make extra unnecessary allocator copies), and is inconsistent with the containers' behavior, which call allocator `construct` on whatever `T` they want. (For example, `std::list<T, alloc<T>>` rebinds to `alloc<_ListNode<T>>`, but calls `construct(T*)` without rebinding back)

It seems like we should be consistent with the containers and not require a rebind here. PR would look something like this, relative to N4810; I'm still not super happy with this wording because it looks like it might be saying a copy of the allocator must be made we would like to avoid…

*[2019-07 Issue Prioritization]*

Priority to 3 after discussion on the reflector.

**Previous resolution [SUPERSEDED]:**

This wording is relative to N4810.

1. Modify 20.3.2.2.7 [util.smartptr.shared.create] as indicated:

> [*Drafting note:* The edits to change `pv` to `pu` were suggested by Jonathan Wakely (thanks!). This wording also has the `remove_cv_t` fixes specified by LWG 3210(i) — if that change is rejected some of those have to be stripped here.]

```
template<class T, ...>
  shared_ptr<T> make_shared(args);
template<class T, class A, ...>
  shared_ptr<T> allocate_shared(const A& a, args);
template<class T, ...>
  shared_ptr<T> make_shared_default_init(args);
template<class T, class A, ...>
  shared_ptr<T> allocate_shared_default_init(const A& a, args);
```

-2- *Requires:* […]

[…]

-7- *Remarks:*

(7.1) — […]

[…]

(7.5) — When a (sub)object of a non-array type `U` is specified to have an initial value of `v`, or `U(l...)`, where `l...` is a list of constructor arguments, `allocate_shared` shall initialize this (sub)object via the expression

(7.5.1) — `allocator_traits<A2>::construct(a2, p~~v~~u, v)` or

(7.5.2) — `allocator_traits<A2>::construct(a2, p~~v~~u, l...)`

respectively, where `p~~v~~u` is a pointer of type `remove_cv_t<U>*` point~~s~~ing to storage suitable to hold an object of type `remove_cv_t<U>` and `a2` of type `A2` is a potentially rebound copy of the allocator `a` passed to `allocate_shared` ~~such that its `value_type` is `remove_cv_t<U>`~~.

(7.6) — […]

(7.7) — When a (sub)object of non-array type `U` is specified to have a default initial value, `allocate_shared` ~~shall~~ initialize~~s~~ this (sub)object via the expression `allocator_traits<A2>::construct(a2, p~~v~~u)`, where `p~~v~~u` is a pointer of type `remove_cv_t<U>*` point~~s~~ing to storage suitable to hold an object of type `remove_cv_t<U>` and `a2` of type `A2` is a potentially rebound copy of the allocator `a` passed to `allocate_shared` ~~such that its `value_type` is `remove_cv_t<U>`~~.

[…]

(7.12) — When a (sub)object of non-array type `U` that was initialized by `allocate_shared` is to be destroyed, it is destroyed via the expression `allocator_traits<A2>::destroy(a2, p~~v~~u)` where `p~~v~~u` is a pointer of type `remove_cv_t<U>*` point~~s~~ing to that object of type `remove_cv_t<U>` and `a2` of type `A2` is a potentially rebound copy of the allocator `a` passed to `allocate_shared` ~~such that its `value_type` is `remove_cv_t<U>`~~.

*[2024-08-23; Jonathan provides updated wording]*

`make_shared_default_init` and `allocate_shared_default_init` were renamed by P1973R1 so this needs a rebase. The edit to (7.11) is just for consistency, so that `pv` is always `void*` and `pu` is `remove_cv_t<U>*`. Accepting this proposed resolution would also resolve issue 3210[(i)].

*[2024-10-02; Reflector poll]*

Set status to Tentatively Ready after six votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4988.

1. Modify 20.3.2.2.7 [util.smartptr.shared.create] as indicated:

```
template<class T, ...>
  shared_ptr<T> make_shared(args);
template<class T, class A, ...>
  shared_ptr<T> allocate_shared(const A& a, args);
template<class T, ...>
  shared_ptr<T> make_shared_for_overwrite(args);
template<class T, class A, ...>
  shared_ptr<T> allocate_shared_for_overwrite(const A& a, args);
```

-2- *Preconditions:* […]

[…]

-7- *Remarks:*

(7.1) — […]

[…]

(7.5) — When a (sub)object of a non-array type `U` is specified to have an initial value of `v`, or `U(l...)`, where `l...` is a list of constructor arguments, `allocate_shared` shall initialize this (sub)object via the expression

(7.5.1) — `allocator_traits<A2>::construct(a2, p~~v~~u, v)` or

(7.5.2) — `allocator_traits<A2>::construct(a2, p~~v~~u, l...)`

respectively, where p~~v~~u is a pointer of type `remove_cv_t<U>*` pointsing to storage suitable to hold an object of type `remove_cv_t<U>` and `a2` of type `A2` is a potentially rebound copy of the allocator `a` passed to `allocate_shared` ~~such that its `value_type` is `remove_cv_t<U>`~~.

(7.6) — […]

(7.7) — When a (sub)object of non-array type `U` is specified to have a default initial value, `allocate_shared` ~~shall~~ initializes this (sub)object via the expression `allocator_traits<A2>::construct(a2, p~~v~~u)`, where p~~v~~u is a pointer of type `remove_cv_t<U>*` pointsing to storage suitable to hold an object of type `remove_cv_t<U>` and `a2` of type `A2` is a potentially rebound copy of the allocator `a` passed to `allocate_shared` ~~such that its `value_type` is `remove_cv_t<U>`~~.

[…]

> [*Drafting note*: Issue 4024[(i)] will add `make_shared_for_overwrite` and `allocate_shared_for_overwrite` to (7.11) but that doesn't conflict with this next edit.]

(7.11) — When a (sub)object of non-array type `U` that was initialized by `make_shared` is to be destroyed, it is destroyed via the expression p~~v~~u->~~v~~U() where p~~v~~u points to that object of type `U`.

(7.12) — When a (sub)object of non-array type `U` that was initialized by `allocate_shared` is to be destroyed, it is destroyed via the expression `allocator_traits<A2>::destroy(a2, p~~v~~u)` where p~~v~~u is a pointer of type `remove_cv_t<U>*` pointsing to that object of type `remove_cv_t<U>` and `a2` of type `A2` is a potentially rebound copy of the allocator `a` passed to `allocate_shared` ~~such that its `value_type` is `remove_cv_t<U>`~~.

---

## 3886. Monad mo' problems

**Section:** 22.5.3.1 [optional.optional.general], 22.8.6.1 [expected.object.general] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2023-02-13 **Last modified:** 2024-09-19

**Priority:** 3

**View other** active issues in [optional.optional.general].

**View all other** issues in [optional.optional.general].

**Discussion:**

While implementing P2505R5 "Monadic Functions for `std::expected`" we found it odd that the template type parameter for the assignment operator that accepts an argument by forwarding reference is defaulted, but the template type parameter for `value_or` is not. For consistency, it would seem that *`meow`*`.value_or(`*`woof`*`)` should accept the same arguments *`woof`* as does *`meow = woof`*, even when those arguments are braced-initializers.

That said, it would be peculiar to default the template type parameter of `value_or` to `T` instead of `remove_cv_t<T>`. For `expected<const vector<int>, int>` *`meow`*`{unexpect, 42};`, for example, *`meow`*`.value_or({1, 2, 3})` would create a temporary `const vector<int>` for the argument and return a copy of that argument. Were the default template argument instead `remove_cv_t<T>`, *`meow`*`.value_or({1, 2, 3})` could move construct its return value from the argument `vector<int>`. For the same reason, the constructor that accepts a forwarding reference with a default template argument of `T` should default that argument to `remove_cv_t<T>`.

For consistency, it would be best to default the template argument of the perfect-forwarding construct, perfect-forwarding assignment operator, and `value_or` to `remove_cv_t<T>`. Since all of the arguments presented apply equally to `optional`, we believe `optional` should be changed consistently with `expected`. MSVCSTL has prototyped these changes successfully.

*[2023-03-22; Reflector poll]*

Set priority to 3 after reflector poll.

*[2024-09-18; Reflector poll]*

Set status to Tentatively Ready after six votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4928.

1. Modify 22.5.3.1 [optional.optional.general] as indicated:

```
namespace std {
  template<class T>
  class optional {
  public:
    […]
    template<class U = remove_cv_t<T>>
      constexpr explicit(see below) optional(U&&);
    […]
    template<class U = remove_cv_t<T>> constexpr optional& operator=(U&&);
    […]
    template<class U = remove_cv_t<T>> constexpr T value_or(U&&) const &;
    template<class U = remove_cv_t<T>> constexpr T value_or(U&&) &&;
```

```
    […]
  };
  […]
}
```

2. Modify 22.5.3.2 [optional.ctor] as indicated:

```
template<class U = remove_cv_t<T>> constexpr explicit(see below) optional(U&& v);
```

-23- *Constraints*: […]

3. Modify 22.5.3.4 [optional.assign] as indicated:

```
template<class U = remove_cv_t<T>> constexpr optional& operator=(U&& v);
```

-12- *Constraints*: […]

4. Modify 22.5.3.7 [optional.observe] as indicated:

```
template<class U = remove_cv_t<T>> constexpr T value_or(U&& v) const &;
```

-15- *Mandates*: […]

[…]

```
template<class U = remove_cv_t<T>> constexpr T value_or(U&& v) &&;
```

-17- *Mandates*: […]

5. Modify 22.8.6.1 [expected.object.general] as indicated:

```
namespace std {
  template<class T, class E>
  class expected {
  public:
    […]
    template<class U = remove_cv_t<T>>
      constexpr explicit(see below) expected(U&& v);
    […]
    template<class U = remove_cv_t<T>> constexpr expected& operator=(U&&);
    […]
    template<class U = remove_cv_t<T>> constexpr T value_or(U&&) const &;
    template<class U = remove_cv_t<T>> constexpr T value_or(U&&) &&;
    […]
  };
  […]
}
```

6. Modify 22.8.6.2 [expected.object.cons] as indicated:

```
template<class U = remove_cv_t<T>>
  constexpr explicit(!is_convertible_v<U, T>) expected(U&& v);
```

-23- *Constraints*: […]

7. Modify 22.8.6.4 [expected.object.assign] as indicated:

```
template<class U = remove_cv_t<T>>
  constexpr expected& operator=(U&& v);
```

-9- *Constraints*: […]

8. Modify 22.8.6.6 [expected.object.obs] as indicated:

```
template<class U = remove_cv_t<T>> constexpr T value_or(U&& v) const &;
```

-16- *Mandates*: […]

[…]

```
template<class U = remove_cv_t<T>> constexpr T value_or(U&& v) &&;
```

-18- *Mandates*: […]

---

## 4084. `std::fixed` ignores `std::uppercase`

**Section:** 28.3.4.3.3.3 [facet.num.put.virtuals] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2024-04-30 **Last modified:** 2024-09-19

**Priority:** 3

**View other** active issues in [facet.num.put.virtuals].

**View all other** issues in [facet.num.put.virtuals].

**Discussion:**

In Table 114 – Floating-point conversions [tab:facet.num.put.fp] we specify that a floating-point value should be printed as if by `%f` when `(flags & floatfield) == fixed`. This ignores whether `uppercase` is also set in `flags`, meaning there is no way to use the conversion specifier `%F` that was added to `printf` in C99.

That's fine for finite values, because 1.23 in fixed format has no exponent character and no hex digits that would need to use uppercase. But `%f` and `%F` are not equivalent for non-finite values, because `%F` prints `"NAN"` and `"INF"` (or `"INFINITY"`). It seems there is no way to print `"NAN"` or `"INF"` using `std::num_put`.

Libstdc++ and MSVC print `"inf"` for the following code, but libc++ prints `"INF"` which I think is non-conforming:

```
std::cout << std::uppercase << std::fixed << std::numeric_limits<double>::infinity();
```

The libc++ behaviour seems more useful and less surprising.

*[2024-05-08; Reflector poll]*

Set priority to 3 after reflector poll. Send to LEWG.

*[2024-09-17; LEWG mailing list vote]*

Set status to Open after LEWG approved the proposed change.

*[2024-09-19; Reflector poll]*

Set status to Tentatively Ready after eight votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4981.

1. Modify 28.3.4.3.3.3 [facet.num.put.virtuals] as indicated:

Table 114 – Floating-point conversions [tab:facet.num.put.fp]

| State | stdio equivalent |
|---|---|
| `floatfield == ios_base::fixed && !uppercase` | `%f` |
| `floatfield == ios_base::fixed` | `%F` |
| `floatfield == ios_base::scientific && !uppercase` | `%e` |
| `floatfield == ios_base::scientific` | `%E` |
| `floatfield == (ios_base::fixed \| ios_base::scientific)` ` && !uppercase` | `%a` |
| `floatfield == (ios_base::fixed \| ios_base::scientific)` | `%A` |
| `!uppercase` | `%g` |
| *otherwise* | `%G` |

---

## 4088. `println` ignores the locale imbued in `std::ostream`

**Section:** 31.7.6.3.5 [ostream.formatted.print] **Status:** Tentatively Ready **Submitter:** Jens Maurer **Opened:** 2024-04-30 **Last modified:** 2024-10-03

**Priority:** Not Prioritized

**View other** active issues in [ostream.formatted.print].

**View all other** issues in [ostream.formatted.print].

**Discussion:**

31.7.6.3.5 [ostream.formatted.print] specifies that `std::print` uses the locale imbued in the `std::ostream&` argument for formatting, by using this equivalence:

```
vformat(os.getloc(), fmt, args);
```

(in the `vformat_(non)unicode` delegation).

However, `std::println` ignores the `std::ostream`'s locale for its locale-dependent formatting:

```
print(os, "{}\n", format(fmt, std::forward<Args>(args)...));
```

This is inconsistent.

*[2024-10-03; Reflector poll]*

Set status to Tentatively Ready after seven votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4981.

1. Modify 31.7.6.3.5 [ostream.formatted.print] as indicated:

```
template<class... Args>
  void println(ostream& os, format_string<Args...> fmt, Args&&... args);
```

-2- *Effects*: Equivalent to:

```
print(os, "{}\n", format(os.getloc(), fmt, std::forward<Args>(args)...));
```

---

## 4113. Disallow `has_unique_object_representations<Incomplete[]>`

**Section:** 21.3.5.4 [meta.unary.prop] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2024-06-25 **Last modified:** 2024-08-02

**Priority:** Not Prioritized

**View other** active issues in [meta.unary.prop].

**View all other** issues in [meta.unary.prop].

**Discussion:**

The type completeness requirements for `has_unique_object_representations` say:

> `T` shall be a complete type, *cv* `void`, or an array of unknown bound.

This implies that the trait works for all arrays of unknown bound, whether the element type is complete or not. That seems to be incorrect, because `has_unique_object_representations_v<Incomplete[]>` is required to have the same result as `has_unique_object_representations_v<Incomplete>` which is ill-formed if `Incomplete` is an incomplete class type.

I think we need the element type to be complete to be able to give an answer. Alternatively, if the intended result for an array of unknown bound is false (maybe because there can be no objects of type `T[]`, or because we can't know that two objects declared as `extern T a[];` and `extern T b[];` have the same number of elements?) then the condition for the trait needs to be special-cased as `false` for arrays of unknown bound. The current spec is inconsistent, we can't allow arrays of unknown bound and apply the current rules to determine the trait's result.

*[2024-08-02; Reflector poll]*

Set status to Tentatively Ready after nine votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4981.

1. Modify 21.3.5.4 [meta.unary.prop] as indicated:

| Template | Condition | Preconditions |
|---|---|---|
| … | … | … |
| `template<class T>`<br>`struct has_unique_object_representations;` | For an array type `T`, the same result as `has_unique_object_representations_v<remove_all_extents_t<T>>`, otherwise *see below*. | `remove_all_extents_t<T>` ~~`T`~~ shall be a complete type~~,~~ or *cv* `void`~~, or an array of unknown bound~~. |

[*Drafting note*: We could use `remove_extent_t<T>` to remove just the first array dimension, because only that first one can have an unknown bound. The proposed resolution uses `remove_all_extents_t<T>` for consistency with the **Condition** column.]

---

## 4119. `generator::promise_type::yield_value(ranges::elements_of<R, Alloc>)`'s nested `generator` may be ill-formed

**Section:** 25.8.5 [coro.generator.promise] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2024-07-11 **Last modified:** 2024-08-02

**Priority:** Not Prioritized

**View other** active issues in [coro.generator.promise].

**View all other** issues in [coro.generator.promise].

**Discussion:**

The nested coroutine is specified to return `generator<yielded, ranges::range_value_t<R>, Alloc>` which can be problematic as the value type of `R` is really irrelevant to `yielded`, unnecessarily violating the `generator`'s *Mandates* (demo):

```
#include <generator>
#include <vector>

std::generator<std::span<int>> f() {
  std::vector<int> v;
  co_yield v; // ok
}

std::generator<std::span<int>> g() {
  std::vector<std::vector<int>> v;
  co_yield std::ranges::elements_of(v); // hard error
}
```

This proposed resolution is to change the second template parameter from `range_value_t<R>` to `void` since that type doesn't matter to us.

*[2024-08-02; Reflector poll]*

Set status to Tentatively Ready after five votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4986.

1. Modify 25.8.5 [coro.generator.promise] as indicated:

```
template<ranges::input_range R, class Alloc>
  requires convertible_to<ranges::range_reference_t<R>, yielded>
  auto yield_value(ranges::elements_of<R, Alloc> r);
```

-13- *Effects*: Equivalent to:

```
auto nested = [](allocator_arg_t, Alloc, ranges::iterator_t<R> i, ranges::sentinel_t<R> s)
  -> generator<yielded, ~~ranges::range_value_t<R>~~void, Alloc> {
    for (; i != s; ++i) {
      co_yield static_cast<yielded>(*i);
    }
  };
return yield_value(ranges::elements_of(nested(
  allocator_arg, r.allocator, ranges::begin(r.range), ranges::end(r.range))));
```

[…]

---

## 4124. Cannot format `zoned_time` with resolution coarser than seconds

**Section:** 30.12 [time.format] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2024-07-26 **Last modified:** 2024-08-02

**Priority:** Not Prioritized

**View other** active issues in [time.format].

**View all other** issues in [time.format].

**Discussion:**

The `std::formatter<std::chrono::zoned_time<Duration, TimeZonePtr>>` specialization calls `tp.get_local_time()` for the object it passes to its base class' `format` function. But `get_local_time()` does not return a `local_time<Duration>`, it returns `local_time<common_type_t<Duration, seconds>>`. The base class' `format` function is only defined for `local_time<Duration>`. That means this is ill-formed, even though the static assert passes:

```
using namespace std::chrono;
  static_assert( std::formattable<zoned_time<minutes>, char> );
  zoned_time<minutes> zt;
  (void) std::format("{}", zt); // error: cannot convert local_time<seconds> to local_time<minutes>
```

Additionally, it's not specified what output you should get for:

```
std::format("{}", local_time_format(zt.get_local_time()));
```

30.12 [time.format] p7 says it's formatted as if by streaming to an `ostringstream`, but there is no `operator<<` for *`local-time-format-t`*. Presumably it should give the same result as `operator<<` for a `zoned_time`, i.e. `"{:L%F %T %Z}"` with padding adjustments etc.

The proposed resolution below has been implemented in libstdc++.

*[2024-08-02; Reflector poll]*

Set status to Tentatively Ready after seven votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4986.

1. Modify 30.12 [time.format] as indicated:

```
template<classDuration, class charT>
  struct formatter<chrono::local-time-format-t<Duration>, charT>;
```

-17- Let `f` be a *`locale-time-format-t`*`<Duration>` object passed to `formatter::format`.

-18- *Remarks*: If the *chrono-specs* is omitted, the result is equivalent to using `%F %T %Z` as the *chrono-specs*. If `%Z` is used, it is replaced with `*f.abbrev` if `f.abbrev` is not a null pointer value. If `%Z` is used and `f.abbrev` is a null pointer value, an exception of type `format_error` is thrown. If `%z` (or a modified variant of `%z`) is used, it is formatted with the value of `*f.offset_sec` if `f.offset_sec` is not a null pointer value. If `%z` (or a modified variant of `%z`) is used and `f.offset_sec` is a null pointer value, then an exception of type `format_error` is thrown.

```
  template<class Duration, class TimeZonePtr, class charT>
  struct formatter<chrono::zoned_time<Duration, TimeZonePtr>, charT>
      : formatter<chrono::local-time-format-t<common_type_t<Duration, seconds>>, charT> {
    template<class FormatContext>
      typename FormatContext::iterator
      format(const chrono::zoned_time<Duration, TimeZonePtr>& tp, FormatContext& ctx) const;
  };

template<class FormatContext>
  typename FormatContext::iterator
```

```
format(const chrono::zoned_time<Duration, TimeZonePtr>& tp, FormatContext& ctx) const;
```

-19- *Effects*: Equivalent to:

```
sys_info info = tp.get_info();
return formatter<chrono::local-time-format-t<common_type_t<Duration, seconds>>, charT>::
         format({tp.get_local_time(), &info.abbrev, &info.offset}, ctx);
```

---

## 4126. Some feature-test macros for fully freestanding features are not yet marked freestanding

**Section:** 17.3.2 [version.syn] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2024-07-24 **Last modified:** 2024-08-02

**Priority:** 2

**View other** active issues in [version.syn].

**View all other** issues in [version.syn].

**Discussion:**

Currently (N4986), it's a bit weird in 17.3.2 [version.syn] that some feature-test macros are not marked freestanding, despite the indicated features being fully freestanding. The freestanding status seems sometimes implicitly covered by "also in" headers that are mostly or all freestanding, but sometimes not.

I think it's more consistent to ensure feature-test macros for fully freestanding features are also freestanding.

*[2024-08-02; Reflector poll]*

Set priority to 2 and set status to Tentatively Ready after seven votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4986.

1. Modify 17.3.2 [version.syn] as indicated:

> [*Drafting note*: `<charconv>` is not fully freestanding, but all functions made `constexpr` by P2291R3 are furtherly made freestanding by P2338R4. ]

```
[…]
#define __cpp_lib_common_reference                 202302L // freestanding, also in <type_traits>
#define __cpp_lib_common_reference_wrapper         202302L // freestanding, also in <functional>
[…]
#define __cpp_lib_constexpr_charconv               202207L // freestanding, also in <charconv>
[…]
#define __cpp_lib_coroutine                        201902L // freestanding, also in <coroutine>
[…]
#define __cpp_lib_is_implicit_lifetime             202302L // freestanding, also in <type_traits>
[…]
#define __cpp_lib_is_virtual_base_of               202406L // freestanding, also in <type_traits>
[…]
#define __cpp_lib_is_within_lifetime               202306L // freestanding, also in <type_traits>
[…]
#define __cpp_lib_mdspan                           202406L // freestanding, also in <mdspan>
[…]
#define __cpp_lib_ratio                            202306L // freestanding, also in <ratio>
[…]
#define __cpp_lib_span_initializer_list            202311L // freestanding, also in <span>
[…]
#define __cpp_lib_submdspan                        202403L // freestanding, also in <mdspan>
[…]
#define __cpp_lib_to_array                         201907L // freestanding, also in <array>
[…]
```

---

## 4135. The helper lambda of `std::erase` for `list` should specify return type as `bool`

**Section:** 23.3.7.7 [forward.list.erasure], 23.3.9.6 [list.erasure] **Status:** Tentatively Ready **Submitter:** Hewill Kang **Opened:** 2024-08-07 **Last modified:** 2024-08-21

**Priority:** Not Prioritized

**Discussion:**

`std::erase` for `list` is specified to return `erase_if(c, [&](auto& elem) { return elem == value; })`. However, the template parameter `Predicate` of `erase_if` only requires that the type of `decltype(pred(...))` satisfies *`boolean-testable`*, i.e., the return type of `elem == value` is not necessarily `bool`.

This means it's worth explicitly specifying the lambda's return type as `bool` to avoid some pedantic cases (demo):

```
#include <list>

struct Bool {
  Bool(const Bool&) = delete;
```

```
  operator bool() const;
};

struct Int {
  Bool& operator==(Int) const;
};

int main() {
  std::list<Int> l;
  std::erase(l, Int{}); // unnecessary hard error
}
```

*[2024-08-21; Reflector poll]*

Set status to Tentatively Ready after nine votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4988.

1. Modify 23.3.7.7 [forward.list.erasure] as indicated:

```
template<class T, class Allocator, class U = T>
  typename forward_list<T, Allocator>::size_type
    erase(forward_list<T, Allocator>& c, const U& value);
```

-1- *Effects*: Equivalent to: `return erase_if(c, [&](const auto& elem) -> bool { return elem == value; });`

2. Modify 23.3.9.6 [list.erasure] as indicated:

```
template<class T, class Allocator, class U = T>
  typename list<T, Allocator>::size_type
    erase(list<T, Allocator>& c, const U& value);
```

-1- *Effects*: Equivalent to: `return erase_if(c, [&](const auto& elem) -> bool { return elem == value; });`

---

## 4140. Useless default constructors for bit reference types

**Section:** 22.9.2.1 [template.bitset.general], 23.3.12.1 [vector.bool.pspc] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2024-08-21 **Last modified:** 2024-09-18

**Priority:** Not Prioritized

**Discussion:**

The standard shows a private default constructor for `bitset<N>::reference` but does not define its semantics, and nothing in the spec refers to it. It was present in C++98, then in C++11 it got `noexcept` added to it, and in C++23 it was made `constexpr` by P2417R2. That's quite a lot of churn for an unusuable member function with no definition.

In libstdc++ it's declared as private, but never defined. In libc++ it doesn't exist at all. In MSVC it is private and defined (and presumably used somewhere). There's no reason for the standard to declare it. Implementers can define it as private if they want to, or not. The spec doesn't need to say anything for that to be true. We can also remove the friend declaration, because implementers know how to do that too.

I suspect it was added as private originally so that it didn't look like `reference` should have an implicitly-defined default constructor, which would have been the case in previous standards with no other constructors declared. However, C++20 added `reference(const reference&) = default;` which suppresses the implicit default constructor, so declaring the default constructor as private is now unnecessary.

Jiang An pointed out in an editorial pull request that `vector<bool, Alloc>::reference` has exactly the same issue.

*[2024-09-18; Reflector poll]*

Set status to Tentatively Ready after eight votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4988.

1. Modify 22.9.2.1 [template.bitset.general] as indicated:

```
namespace std {
  template<size_t N> class bitset {
  public:
    // bit reference
    class reference {
      friend class bitset;
      constexpr reference() noexcept;

    public:
      constexpr reference(const reference&) = default;
      constexpr ~reference();
      constexpr reference& operator=(bool x) noexcept;            // for b[i] = x;
      constexpr reference& operator=(const reference&) noexcept;  // for b[i] = b[j];
      constexpr bool operator~() const noexcept;                  // flips the bit
      constexpr operator bool() const noexcept;                   // for x = b[i];
```

```
    constexpr reference& flip() noexcept;                       // for b[i].flip();
  };
```

2. Modify 23.3.12.1 [vector.bool.pspc], `vector<bool, Allocator>` synopsis, as indicated:

```
namespace std {
  template<class Allocator>
  class vector<bool, Allocator> {
  public:
    // types
    […]
    // bit reference
    class reference {
      friend class vector;
      constexpr reference() noexcept;

    public:
      constexpr reference(const reference&) = default;
      constexpr ~reference();
      constexpr operator bool() const noexcept;
      constexpr reference& operator=(bool x) noexcept;
      constexpr reference& operator=(const reference& x) noexcept;
      constexpr const reference& operator=(bool x) const noexcept;
      constexpr void flip() noexcept;   // flips the bit
    };
```

---

## 4141. Improve prohibitions on "additional storage"

**Section:** 22.5.3.1 [optional.optional.general], 22.6.3.1 [variant.variant.general], 22.8.6.1 [expected.object.general], 22.8.7.1 [expected.void.general] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2024-08-22 **Last modified:** 2024-09-18

**Priority:** Not Prioritized

**View other** active issues in [optional.optional.general].

**View all other** issues in [optional.optional.general].

**Discussion:**

This issue was split out from issue 4015[(i)].

`optional`, `variant` and `expected` all use similar wording to require their contained value to be a subobject, rather than dynamically allocated and referred to by a pointer, e.g.

> When an instance of `optional<T>` contains a value, it means that an object of type `T`, referred to as the optional object's *contained value*, is allocated within the storage of the optional object. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate its contained value.

During the LWG reviews of P2300 in St. Louis, concerns were raised about the form of this wording and whether it's normatively meaningful. Except for the special case of standard-layout class types, the standard has very few requirements on where or how storage for subobjects is allocated. The library should not be trying to dictate more than the language guarantees. It would be better to refer to wording from 6.7.2 [intro.object] such as *subobject*, *provides storage*, or *nested within*. Any of these terms would provide the desired properties, without using different (and possibly inconsistent) terminology.

Using an array of bytes to *provide storage* for the contained value would make it tricky to meet the constexpr requirements of types like `optional`. This means in practice, the most restrictive of these terms, *subobject*, is probably accurate and the only plausible implementation strategy. However, I don't see any reason to outlaw other implementation strategies that might be possible in future (say, with a constexpr type cast, or non-standard compiler-specific instrinics). For this reason, the proposed resolution below uses *nested within*, which provides the desired guarantee without imposing additional restrictions on implementations.

*[2024-09-18; Reflector poll]*

Set status to Tentatively Ready after seven votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4988.

1. Modify 22.5.3.1 [optional.optional.general] as indicated:

> [*Drafting note*: This edit modifies the same paragraph as issue 4015[(i)], but that other issue intentionally doesn't touch the affected sentence here (except for removing the italics on "contained value"). The intention is that the merge conflict can be resolved in the obvious way: "An optional object's contained value is nested within (6.7.2 [intro.object]) the optional object."]

> -1- Any instance of `optional<T>` at any given time either contains a value or does not contain a value. When an instance of `optional<T>` *contains a value*, it means that an object of type `T`, referred to as the optional object's *contained value*, is ~~allocated within the storage of~~ nested within (6.7.2 [intro.object]) the optional object. ~~Implementations are not permitted to use additional storage, such as dynamic memory, to allocate its contained value.~~ When an object of type `optional<T>` is contextually converted to `bool`, the conversion returns `true` if the object contains a value; otherwise the conversion returns `false`.

2. Modify 22.6.3.1 [variant.variant.general] as indicated:

> -1- Any instance of `variant` at any given time either holds a value of one of its alternative types or holds no value. When an instance of `variant` holds a value of alternative type `T`, it means that a value of type `T`, referred to as the `variant` object's *contained value*, is ~~allocated within the~~

~~storage of~~ nested within (6.7.2 [intro.object]) the `variant` object. ~~Implementations are not permitted to use additional storage, such as dynamic memory, to allocate the contained value.~~

3. Modify 22.8.6.1 [expected.object.general] as indicated:

> -1- Any object of type `expected<T, E>` either contains a value of type `T` or a value of type `E` ~~within its own storage~~ nested within (6.7.2 [intro.object]) it. ~~Implementations are not permitted to use additional storage, such as dynamic memory, to allocate the object of type `T` or the object of type `E`.~~ Member *`has_val`* indicates whether the `expected<T, E>` object contains an object of type `T`.

4. Modify 22.8.7.1 [expected.void.general] as indicated:

> -1- Any object of type `expected<T, E>` either represents a value of type `T`, or contains a value of type `E` ~~within its own storage~~ nested within (6.7.2 [intro.object]) it. ~~Implementations are not permitted to use additional storage, such as dynamic memory, to allocate the object of type `E`.~~ Member *`has_val`* indicates whether the `expected<T, E>` represents a value of type `T`.

---

## 4142. `format_parse_context::check_dynamic_spec` should require at least one type

**Section:** 28.5.6.6 [format.parse.ctx] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2024-08-28 **Last modified:** 2024-09-18

**Priority:** Not Prioritized

**View all other** issues in [format.parse.ctx].

**Discussion:**

The *Mandates*: conditions for `format_parse_context::check_dynamic_spec` are:

> -14- *Mandates*: The types in `Ts...` are unique. Each type in `Ts...` is one of `bool`, `char_type`, `int`, `unsigned int`, `long long int`, `unsigned long long int`, `float`, `double`, `long double`, `const char_type*`, `basic_string_view<char_type>`, or `const void*`.

There seems to be no reason to allow `Ts` to be an empty pack, that's not useful. There is no valid arg-id value that can be passed to it if the list of types is empty, since `arg(n)` will never be one of the types in an empty pack. So it's never a constant expression if the pack is empty.

*[2024-09-18; Reflector poll]*

Set status to Tentatively Ready after seven votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4988.

1. Modify 28.5.6.6 [format.parse.ctx] as indicated:

```
template<class... Ts>
  constexpr void check_dynamic_spec(size_t id) noexcept;
```

> -14- *Mandates*: `sizeof...(Ts) > 1`. The types in `Ts...` are unique. Each type in `Ts...` is one of `bool`, `char_type`, `int`, `unsigned int`, `long long int`, `unsigned long long int`, `float`, `double`, `long double`, `const char_type*`, `basic_string_view<char_type>`, or `const void*`.
>
> -15- *Remarks*: A call to this function is a core constant expression only if:
>
> (15.1) — `id < num_args_` is `true` and
> (15.2) — the type of the corresponding format argument (after conversion to `basic_format_arg<Context>`) is one of the types in `Ts...`.

---

## 4144. Disallow `unique_ptr<T&, D>`

**Section:** 20.3.1.3.1 [unique.ptr.single.general] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2024-08-30 **Last modified:** 2024-11-13

**Priority:** Not Prioritized

**View all other** issues in [unique.ptr.single.general].

**Discussion:**

It seems that we currently allow nonsensical specializations of `unique_ptr` such as `unique_ptr<int&, D>` and `unique_ptr<void()const, D>` (a custom deleter that defines `D::pointer` is needed, because otherwise the `pointer` type would default to invalid types like `int&*` or `void(*)()const`). There seems to be no reason to support these "unique pointer to reference" and "unique pointer to abominable function type" specializations, or any specialization for a type that you couldn't form a raw pointer to.

Prior to C++17, the major library implementations rejected such specializations as a side effect of the constraints for the `unique_ptr(auto_ptr<U>&&)` constructor being defined in terms of `is_convertible<U*, T*>`. This meant that overload resolution for any constructor of `unique_ptr` would attempt to form the type `T*` and fail if that was invalid. With the removal of `auto_ptr` in C++17, that constructor was removed and now `unique_ptr<int&, D>` can be instantiated (assuming any zombie definition of `auto_ptr` is not enabled by the library). This wasn't intentional, but just an accident caused by not explicitly forbidding such types.

Discussion on the LWG reflector led to near-unanimous support for explicitly disallowing these specializations for non-pointable types.

*[2024-11-13; Reflector poll]*

Set status to Tentatively Ready after eight votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4988.

1. Modify 20.3.1.3.1 [unique.ptr.single.general] as indicated:

> -?- A program that instantiates the definition of `unique_ptr<T, D>` is ill-formed if `T*` is an invalid type.
> [*Note*: This prevents the intantiation of specializations such as `unique_ptr<T&, D>` and `unique_ptr<int() const, D>`. — *end note*]
>
> -1- The default type for the template parameter `D` is `default_delete`. A client-supplied template argument `D` shall be a function object type (22.10 [function.objects]), lvalue reference to function, or lvalue reference to function object type for which, given a value `d` of type `D` and a value `ptr` of type `unique_ptr<T, D>::pointer`, the expression `d(ptr)` is valid and has the effect of disposing of the pointer as appropriate for that deleter.
>
> -2- If the deleter's type `D` is not a reference type, `D` shall meet the *Cpp17Destructible* requirements (Table 35).
>
> -3- If the *qualified-id* `remove_reference_t<D>::pointer` is valid and denotes a type (13.10.3 [temp.deduct]), then `unique_ptr<T, D>::pointer` shall be a synonym for `remove_reference_t<D>::pointer`. Otherwise `unique_ptr<T, D>::pointer` shall be a synonym for `element_type*`. The type `unique_ptr<T, D>::pointer` shall meet the *Cpp17NullablePointer* requirements (Table 36).
>
> -4- [*Example 1*: Given an allocator type `X` (16.4.4.6.1 [allocator.requirements.general]) and letting `A` be a synonym for `allocator_traits<X>`, the types `A::pointer`, `A::const_pointer`, `A::void_pointer`, and `A::const_void_pointer` may be used as `unique_ptr<T, D>::pointer`. — *end example*]

---

## 4147. Precondition on `inplace_vector::emplace`

**Section:** 23.2.4 [sequence.reqmts] **Status:** Tentatively Ready **Submitter:** Arthur O'Dwyer **Opened:** 2024-08-26 **Last modified:** 2024-09-18

**Priority:** Not Prioritized

**View other** active issues in [sequence.reqmts].

**View all other** issues in [sequence.reqmts].

**Discussion:**

Inserting into the middle of an `inplace_vector`, just like inserting into the middle of a `vector` or `deque`, requires that we construct the new element out-of-line, shift down the trailing elements (*Cpp17MoveAssignable*), and then move-construct the new element into place (*Cpp17MoveInsertable*). P0843R14 failed to make this change, but it should have.

*[2024-09-18; Reflector poll]*

Set status to Tentatively Ready after nine votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4988.

1. Modify 23.2.4 [sequence.reqmts] as indicated:

> `a.emplace(p, args)`
>
> -19- *Result*: `iterator`.
>
> -20- *Preconditions*: `T` is *Cpp17EmplaceConstructible* into `X` from `args`. For `vector`, `inplace_vector`, and `deque`, `T` is also *Cpp17MoveInsertable* into `X` and *Cpp17MoveAssignable*.
>
> -21- *Effects*: Inserts an object of type `T` constructed with `std::forward<Args>(args)...` before `p`.
>
> [*Note 1*: `args` can directly or indirectly refer to a value in `a`. — *end note*]
>
> -22- *Returns*: An iterator that points to the new element constructed from `args` into `a`.

---

## 4148. `unique_ptr::operator*` should not allow dangling references

**Section:** 20.3.1.3.5 [unique.ptr.single.observers] **Status:** Tentatively Ready **Submitter:** Jonathan Wakely **Opened:** 2024-09-02 **Last modified:** 2024-09-18

**Priority:** Not Prioritized

**View other** active issues in [unique.ptr.single.observers].

**View all other** issues in [unique.ptr.single.observers].

**Discussion:**

If `unique_ptr<T,D>::element_type*` and `D::pointer` are not the same type, it's possible for `operator*()` to return a dangling reference that has undefined behaviour.

```
struct deleter {
  using pointer = long*;
  void operator()(pointer) const {}
};
long l = 0;
std::unique_ptr<const int, deleter> p(&l);
int i = *p; // undefined
```

We should make this case ill-formed.

*[2024-09-18; Reflector poll]*

Set status to Tentatively Ready after seven votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4988.

1. Modify 20.3.1.3.5 [unique.ptr.single.observers] as indicated:

```
constexpr add_lvalue_reference_t<T> operator*() const noexcept(noexcept(*declval<pointer>()));
```

-?- *Mandates*: `reference_converts_from_temporary_v<add_lvalue_reference_t<T>, decltype(*declval<pointer>())>` is `false`.

-1- *Preconditions*: `get() != nullptr` is `true`.

-2- *Returns*: `*get()`.

---

## 4153. Fix extra "-1" for `philox_engine::max()`

**Section:** 29.5.4.5 [rand.eng.philox] **Status:** Tentatively Ready **Submitter:** Ruslan Arutyunyan **Opened:** 2024-09-18 **Last modified:** 2024-10-02

**Priority:** Not Prioritized

**View other** active issues in [rand.eng.philox].

**View all other** issues in [rand.eng.philox].

**Discussion:**

There is a typo in `philox_engine` wording that makes "-1" two times instead of one for `max()` method. The reason for that typo is that the wording was originally inspired by `mersenne_twister_engine` but after getting feedback that what is written in the `philox_engine` synopsis is not C++ code, the authors introduced the *m* variable (as in `subtract_with_carry_engine`) but forgot to remove "-1" in the *m* definition.

Note: after the proposed resolution below is applied the *m* variable could be reused in other places: basically in all places where the `mod 2^w` pattern appears (like `subtract_with_carry_engine` does). The authors don't think it's worth changing the rest of the wording to reuse the *m* variable. If somebody thinks otherwise, please provide such feedback.

*[2024-10-02; Reflector poll]*

Set status to Tentatively Ready after six votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4988.

1. Modify 29.5.4.5 [rand.eng.philox] as indicated:

-1- A `philox_engine` random number engine produces unsigned integer random numbers in the ~~closed~~ interval [0, $m$~~-1~~), where $m = 2^w$ ~~- 1~~ and the template parameter $w$ defines the range of the produced numbers.

---

## 4157. The resolution of LWG3465 was damaged by P2167R3

**Section:** 17.11.6 [cmp.alg] **Status:** Tentatively Ready **Submitter:** Jiang An **Opened:** 2024-09-18 **Last modified:** 2024-10-02

**Priority:** Not Prioritized

**View other** active issues in [cmp.alg].

**View all other** issues in [cmp.alg].

**Discussion:**

In the resolution of LWG 3465[(i)], `F < E` was required to be well-formed and implicitly convertible to `bool`. However, P2167R3 replaced the convertibility requirements with just "each of `decltype(E == F)` and `decltype(E < F)` models *`boolean-testable`*", which rendered the type of `F < E` underconstrained.

*[2024-10-02; Reflector poll]*

Set status to Tentatively Ready after seven votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4988.

1. Modify 17.11.6 [cmp.alg] as indicated:

> (6.3) — Otherwise, if the expressions `E == F`, `E < F`, and `F < E` are all well-formed and each of `decltype(E == F)` ~~and~~, `decltype(E < F)` , and `decltype(F < E)` models *`boolean-testable`*,
>
> ```
> E == F ? partial_ordering::equivalent :
> E < F  ? partial_ordering::less :
> F < E  ? partial_ordering::greater :
>          partial_ordering::unordered
> ```
>
> except that `E` and `F` are evaluated only once.

---

## 4169. `std::atomic<T>`'s default constructor should be constrained

**Section:** 32.5.8.2 [atomics.types.operations] **Status:** Tentatively Ready **Submitter:** Giuseppe D'Angelo **Opened:** 2024-10-15 **Last modified:** 2024-11-13

**Priority:** Not Prioritized

**View other** active issues in [atomics.types.operations].

**View all other** issues in [atomics.types.operations].

**Discussion:**

The current wording for `std::atomic`'s default constructor in 32.5.8.2 [atomics.types.operations] specifies:

```
constexpr atomic() noexcept(is_nothrow_default_constructible_v<T>);
```

> *Mandates*: `is_default_constructible_v<T>` is `true`.

This wording has been added by P0883R2 for C++20, which changed `std::atomic`'s default constructor to always value-initialize. Before, the behavior of this constructor was not well specified (this was LWG issue 2334[(i)]).

The usage of a *Mandates* element in the specification has as a consequence that `std::atomic<T>` is always default constructible, even when `T` is not. For instance:

```
// not default constructible:
struct NDC { NDC(int) {} };

static_assert(std::is_default_constructible<std::atomic<NDC>>); // OK
```

The above check is OK as per language rules, but this is user-hostile: actually using `std::atomic<NDC>`'s default constructor results in an error, despite the detection saying otherwise.

Given that `std::atomic<T>` already requires `T` to be complete anyhow (32.5.8.1 [atomics.types.generic.general] checks for various type properties which require completeness) it would be more appropriate to use a constraint instead, so that `std::atomic<T>` is default constructible if and only if `T` also is.

*[2024-11-13; Reflector poll]*

Set status to Tentatively Ready after seven votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4993.

1. Modify 32.5.8.2 [atomics.types.operations] as indicated:

> [*Drafting note*: There is implementation divergence at the moment; libstdc++ already implements the proposed resolution and has done so for a while.]
>
> ```
> constexpr atomic() noexcept(is_nothrow_default_constructible_v<T>);
> ```
>
> -1- *Constraints*~~*Mandates*~~: `is_default_constructible_v<T>` is `true`.
>
> -2- *Effects*: […]

---

## 4170. `contiguous_iterator` should require `to_address(I{})`

**Section:** 24.3.4.14 [iterator.concept.contiguous] **Status:** Tentatively Ready **Submitter:** Casey Carter **Opened:** 2024-11-01 **Last modified:** 2024-11-13

**Priority:** Not Prioritized

**View all other** issues in [iterator.concept.contiguous].

**Discussion:**

The design intent of the `contiguous_iterator` concept is that iterators can be converted to pointers denoting the same sequence of elements. This enables a common range `[i, j)` or counted range `i + [0, n)` to be processed with extremely efficient low-level C or assembly code that operates on `[to_address(i), to_address(j))` (respectively `to_address(i) + [0, n)`).

A value-initialized iterator `I{}` can be used to denote the empty ranges `[I{}, I{})` and `I{} + [0, 0)`. While the existing semantic requirements of `contiguous_iterator` enable us to convert both dereferenceable and past-the-end iterators with `to_address`, converting ranges involving value-initialized iterators to pointer ranges additionally needs `to_address(I{})` to be well-defined. Note that `to_address` is already implicitly equality-preserving for `contiguous_iterator` arguments. Given this additional requirement `to_address(I{}) == to_address(I{})` and `to_address(I{}) == to_address(I{}) + 0` both hold, so the two types of empty ranges involving value-initialized iterators convert to empty pointer ranges as desired.

*[2024-11-13; Reflector poll]*

Set status to Tentatively Ready after eight votes in favour during reflector poll.

**Proposed resolution:**

This wording is relative to N4993.

1. Modify 24.3.4.14 [iterator.concept.contiguous] as indicated:

-1- The `contiguous_iterator` concept provides a guarantee that the denoted elements are stored contiguously in memory.

```
template<class I>
  concept contiguous_iterator =
    random_access_iterator<I> &&
    derived_from<ITER_CONCEPT(I), contiguous_iterator_tag> &&
    is_lvalue_reference_v<iter_reference_t<I>> &&
    same_as<iter_value_t<I>, remove_cvref_t<iter_reference_t<I>>> &&
    requires(const I& i) {
      { to_address(i) } -> same_as<add_pointer_t<iter_reference_t<I>>>;
    };
```

-2- Let `a` and `b` be dereferenceable iterators and `c` be a non-dereferenceable iterator of type `I` such that `b` is reachable from `a` and `c` is reachable from `b`, and let `D` be `iter_difference_t<I>`. The type `I` models `contiguous_iterator` only if

(2.1) — `to_address(a) == addressof(*a)`,

(2.2) — `to_address(b) == to_address(a) + D(b - a)`,

(2.3) — `to_address(c) == to_address(a) + D(c - a)`,

(2.?) — `to_address(I{})` is well-defined,

(2.4) — `ranges::iter_move(a)` has the same type, value category, and effects as `std::move(*a)`, and

(2.5) — if `ranges::iter_swap(a, b)` is well-formed, it has effects equivalent to `ranges::swap(*a, *b)`.